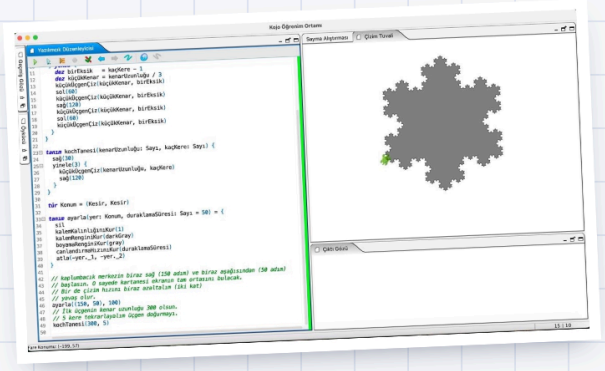
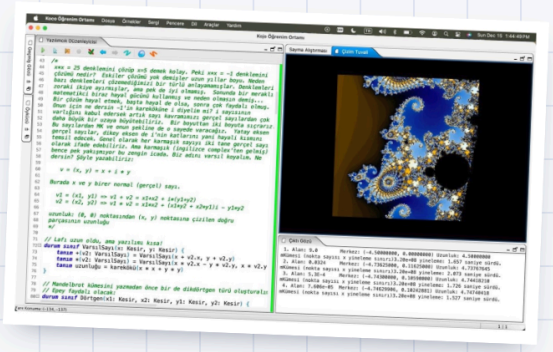
FEN LİSESİ İÇİN KOCO VE SCALA

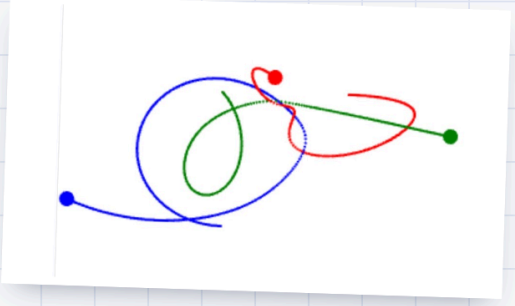
Programlamaya ve Algoritmalara Keyifli ve İşlevsel Bir Giriş

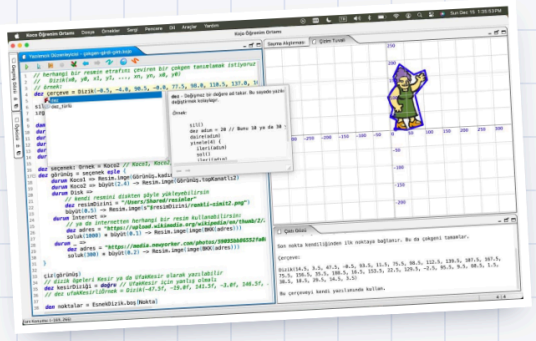
Altı bölüm, bir kaplumbağa, bol fraktal ve bir sürü çizim.
Her satırın sonucunu anında ekranda görün.

I İlk Adımlar · II Veri · III Soyutlama · IV Özyineleme · V Çizgeler · VI Dinamik Programlama









hepsi Koco'da yazılmış
programların çıktısı

Bülent Başaran

2026 · ilk taslak

CC BY-SA 4.0 · kod MIT

github.com/bulent2k2/cpp_ogreniyoruz

Programlamaya ve Algoritmalara Keyifli ve İşlevsel Bir Giriş

Bu kitapçık, C++ dili üzerine kurulu *Keyifli Bir Başlangıç*'ın kardeşi: aynı kavramlar, bambaşka bir ortam. Bu sefer derleyici beklemek yok — yazdığınız her satırın sonucunu *anında, ekranda, çizim olarak* göreceksiniz. Ve yol boyunca programlamanın en zarif üslubuyla tanışacaksınız: *işlevsel* üslupla.

I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

ÖNSÖZ

Merhaba

Derslerimizde C++ öğrenirken ara sıra başka bir pencere açıp size bir şeyler gösterdim: kendini çizen kar taneleri, sonsuz derinlikte desenler, birbirinin çevresinde dans eden gök cisimleri. O pencerenin adı Koco — ve bu kitapçık, o pencereyi sonuna kadar açıyor.

Koco, Scala diliyle çalışan, tamamı Türkçeleştirilmiş bir öğrenme ortamı. Komutları Türkçe, türleri Türkçe, yardım sayfaları Türkçe — ve yamalı Scala derleyicisi sayesinde dilin *anahtar sözcükleri bile* Türkçe. Bunu başka hiçbir dilin Kojo'su yapmıyor. Birkaç iş arkadaşımla yıllardır gönüllü olarak emek verdiğimiz açık kaynak bir proje; kısacası biraz da bizim çocuk. Ama onu bu kitapçığa taşıyan şey duygusallık değil, iki somut üstünlük:

1. **Anında geri bildirim.** Kod yazarsınız, çalıştırırsınız, *o saniye* ekranda bir şey olur. Derleme komutu yok, terminal yok, kurulum derdi neredeyse yok. Öğrenmenin en kısa döngüsü budur: dene, gör, değiştir, yine dene.

2. **İşlevsel üslup.** Scala, *işlevsel programlamanın* (*functional programming*) en güçlü dillerinden. Bu üslupta programlar değişkenleri değiştire değiştire değil, değerleri *dönüştüre dönüştüre* ilerler. Kod kısalır, hata azalır, kanıt kolaylaşır. C++ yazarken de bir daha eskisi gibi yazamazsınız — iyi anlamda.

“İşlevsel” sözcüğü şimdilik havada kalabilir; öyle kalsın. Bu kitapçığın işi onu adım adım elle tutulur yapmak. Altı bölümün sonunda dönüp bu sayfaya baktığınızda, iki anlamını da görmüş olacaksınız: hem *işlev*lerle örülü, hem de gerçekten işe yarar.

KARDEŞLİK

İki kitapçık, tek yol

Bu kitapçığın bir kardeşi var: Programlamaya ve Algoritmalara Keyifli Bir Başlangıç — aynı konuları C++ ile anlatıyor. Bölümleri bilerek paralel kurdum: orada sayı tahmin oyunu varsa burada da var; orada özyineleme Hanoi kuleleriyle geliyorsa burada Koch tanesiyle geliyor; oradaki çizge gezisi burada işlevsel kılıkta karşınıza çıkıyor.

Hangisinden başlamalı? Fark etmez; ikisi de kendi başına okunuyor. Ama asıl güzeli ikisini **yan yana** okumak. Aynı algoritmayı iki dilde görmek, tek dilde iki kere görmekten çok daha öğretici — tıpkı iki farklı çözümü karşılaştırmanın tek çözümü ezberlemekten öğretici olması gibi.

> **PÜF NOKTASI**
>
> Bir bölümü burada bitirince kardeş kitapçıktaki eşine göz atın ve kendinize şu soruyu sorun: *hangi kısım orada daha kolaydı, hangisi burada?* Bu sorunun yanıtı size dillerin karakterini, dillerin karakteri de programlamanın kendisini öğretir.

TEZGÂH

Koco'yu kurun

Tezgâh bu sefer çok kısa; Koco tek parça bir uygulama.

1. Koco sürüm sayfasından işletim sisteminize uygun paketi indirin ve kurun. Windows, macOS ve Linux'ta çalışıyor.
2. Koco'yu açın. Karşınıza üç bölge çıkacak: üstte **tuval** (çizimlerin yaşadığı yer), altta **yazılımcık düzenleyicisi** (kodunuzu yazdığınız yer), sağda **çıktı penceresi**. “Kaplumbağalı Kojo'ya Hoşgeldin!” yazısını gördüyseniz hazırsınız.
3. İlk komutunuzu yazın ve çalıştırma düğmesine basın: `ileri(100)`. Tuvalde yukarı doğru bir çizgi belirecek. Çizen şey bir *kaplumbağa* — birazdan tanışacaksınız.

> **DENEYİN**
>
> Kurulum istemeyenler için tarayıcıda çalışan Türkçe bir sürüm var: ikojo.fly.dev. Düğmeleri de Türkçe: kodu yapıştırın, **Çalıştır**'a basın. Bu kitapçıktaki bütün çizim örnekleri orada çalıştırıldı; görseller o çıktıların ekran görüntüleri. (İngilizce aslı ikojo.in.)

İKİ KOCO, ÜÇ FARK

Masaüstü Koco ile tarayıcıdaki iKojo aynı dili konuşur; üç yerde ayrılırlar, bilin de şaşırmayın:

- **Yazdırma biçimi.** iKojo bir dizini `List(1, 2, 3)`, bir ikili `true` diye basar; masaüstü Koco `Dizin(1, 2, 3)` ve `doğru` der. Alıştırma defteri bu yüzden dizinleri `yazıYap` ile, ikilleri `evet/hayır` diye yazdırır.
- **Birkaç komut yalnız masaüstünde.** `silVeSakla` gibi pencereyi yöneten komutlar iKojo'da yok; `sil()` ve `çıktıyıSil()` ikisinde de var.
- **Girdi pencereleri.** `satıroku` gibi girdi isteyen komutlar masaüstünde küçük bir pencere açar; o pencere çıktı alanını örtebilir. Kenara çekin, çıktı altında duruyordur. iKojo'da girdi doğrudan sayfada sorulur.

Bir de yol arkadaşı kaynaklar: iKojo'nun içindeki Türkçe Skala kılavuzu ile komut kılavuzu, ve ders depomuzdaki Koco'ya davet yazısı. Takıldığınız her komut için Koco'nun içinde de yardım var: komutun üstüne gelip bekleyin ya da `Ctrl+Boşluk` ile kod tamamlamayı açın.

YÖNTEM

Nasıl çalışalım?

Kardeş kitapçıktaki altı ilkenin hepsi burada da geçerli; özellikle üçünü hatırlatıp geçiyorum:

1. **Çalıştırmadan inanmayın.** Koco'da bunun bahanesi hiç yok: çalıştırmak tek tuş, sonuç bir saniye sonra gözünüzün önünde. Her kod parçasını görür görmez çalıştırın; sonra bir yerini değiştirip *bir daha* çalıştırın.
2. **İkili çalışın.** Biri yazar, öteki bir adım ileriyi düşünür; on beş dakikada bir değişin. Çizim programlarında bu bir de şöyle güzel: yol gösterici "şimdi kaplumbağa nereye bakıyor?" diye sorar, ve bu soru hataların yarısını daha yazılmadan yakalar.
3. **Yarım bıraktığım yerler kasıtlı.** Bu kitapçıkta da çözümü verilmemiş sorular, bilerek eksik bırakılmış programlar var. Onlar boşluk değil, davet.

YAPAY ZEKÂ KURALI

Elinizde bir model varsa, kardeş kitapçıkla aynı kural: alıştırma defterinin Isınma ve Alıştırma basamaklarında hiç sormayın; Zorlayıcı'da yalnızca ipucu isteyin, kod istemeyin; Keşif'te serbest, ama yapıştırdığınız her satırı açıklayabilmelisiniz. Koco'da bir de şu var: modelin yazdığı kod çoğu zaman İngilizce adlarla gelir (`forward`, `repeat`); Türkçe karşılığını bilmeden yapıştırmayın.

TAKIM İŞİ

Haftada bir sabit saat belirleyin ve koruyun. Bu kitapçık altı bölüm; her bölüm bir-iki haftalık keyifli bir iş. Bir dönemde, kardeş kitapçıkla birlikte ikisini de bitirmiş olursunuz.

Bu kitapçıkta neler var?

BÖLÜM	KONU	SONUNDA YAPABİLECEĞİNİZ
I. İlk Adımlar	Kaplumbağa komutları; değer, tür, işlev; yineleme	Kendi desenlerinizi çizen programlar ve bir sayı tahmin oyunu
II. Veriyi Düzenlemek	Dizin, küme, eşlem; işle, ele, indirge; Belki	Döngü yazmadan veri işleyen, tek satırlık çözümler
III. Soyutlama	İşlevi girdi alan işlevler, değişmez değerler, sınama, ölçme	Kendi `yinele`'nizi yazmak; kendini sınayan program
IV. Özyineleme	Taban ve adım; ağaçlar, Koch tanesi, Hanoi; bellekle hızlandırma	Özyinelemeyi yazmak değil, *görmek*: kendi fraktallarınız
V. Çizgeler	Çizge temsili; derinlemesine ve enlemesine gezi, işlevsel kılıkta	Değişmez kümelerle çizge gezen, kanıtı kolay kod
VI. Dinamik Programlama	Bellek, ızgara yolları, bozuk paralar; Mandelbrot	Tümevarım formülünü doğrudan koda çevirmek — ve yola devam
Sözlük	Terimlerin Türkçesi, Scala'daki ve İngilizcedeki karşılığı, bir cümlelik anlamı	Koco'nun Türkçe adıyla Scala'nın özgün adını yan yana görmek

SON BİR UYARI

Bu kitapçıktaki **alıştırmaların çözümleri yok**. İpucu var, yanıt yok. Yanıt sizde — ya da yan masadaki arkadaşınızda. Bir de: buradaki bütün kodlar Koco'da çalışacak biçimde yazıldı; birebir yazıp çalıştırın, sonra bozup düzeltin. Kod bozmak serbest, hatta ödev.

> **ANAHTAR SÖZCÜKLER DE TÜRKÇE**
>
> Koco'nun Türkçesi yüzeyde kalmıyor: dilin çekirdek sözcükleri bile Türkçe. Bu kitapçıkta en çok şunları göreceksiniz: `tanım` (*def*), `dez` (*val* — değişmez değer), `den` (*var* — değişken), `eğer` / `yoksa` (*if/else*), `eşle` / `durum` (*match/case*), `tür` (*type*), `durum sınıf` (*case class*), `doğru` / `yanlış` (*true/false*). Parantez içindeki İngilizce özgün adlar da Koco'da aynen çalışır; Scala kitaplarına geçtiğinizde tanış çıkarsınız.

KÜNYE

Telif ve lisans

Programlamaya ve Algoritmalara Keyifli ve İşlevsel Bir Giriş
Bülent Başaran · 2026 · İlk taslak

Bu kitapçığın **metni** Creative Commons Atıf-AynıLisanslaPaylaş 4.0 (CC BY-SA 4.0) lisansıyla sunuluyor; içindeki **örnek programlar** MIT lisansı altında. Kopyalayın, çalıştırın, bozun, düzeltin, paylaşın.

Kojo, Lalit Pant'ın kurduğu ve dünyanın dört bir yanından gönüllülerin katkı verdiği açık kaynak bir projedir; Türkçe sürümü Koco'ya bu satırların yazarı da katkıda bulunuyor. Kitapçıkta anılan dış kaynaklar kendi sahiplerine ve koşullarına tabidir.

Bu kitapçıktaki bütün programlar, Koco'nun Türkçe anahtar sözcüklü Scala derleyicisiyle (`scala-tr`, 2.13.18) derlendi; konsol örnekleri çalıştırıldı ve gösterilen çıktılar gerçek çıktılardır. Çizim örneklerinin görüntüsünü ise kendi Koco'nuzda alacaksınız — zaten bütün mesele o. Bir hata bulursanız ya da bir bölümün daha iyi anlatılabileceğini düşünüyorsanız, ders deposunda konu açın. Kitapçık da ders gibi, birlikte gelişiyor.

BÖLÜM I

Kaplumbağayla İlk Adımlar

Ekranın ortasında sizi bekleyen küçük bir kaplumbağa var. Komut verirseniz yürüyor, yürürken de çiziyor. Bu bölümde ona kareler, yıldızlar ve çiçekler çizdireceğiz — ve farkında olmadan programlamanın bütün yapıtaşlarını öğrenmiş olacaksınız: değer, tür, işlev, yineleme. Sonunda da bilgisayarla bir oyun oynayacağız.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

I.1

Kaplumbağayla tanışın

Koco'yu açın (ya da tarayıcıda ikojo.fly.dev'i), aşağıdaki iki satırı yazın ve çalıştırın:

ilk çizim

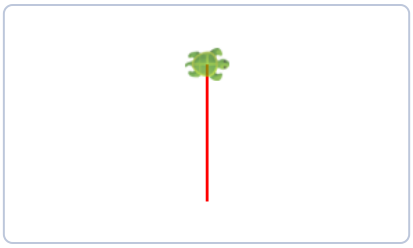
```
ileri(100)     // 100 adım ileri
sağ()          // 90 derece sağa dön
```

Tuvalde bir çizgi belirdi ve kaplumbağa artık doğuya bakıyor. Hepsi bu: kaplumbağa *baktığı yöne* yürür, yürürken kalemiyle iz bırakır. Şimdi bir kare çizelim. En kestirme yol, dört kere “ileri ve dön” demek:

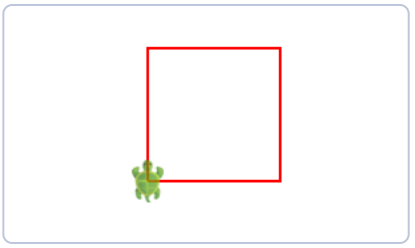
kare — İLK YİNELEME

```
yinele(4) {
  ileri(100)
  sağ()
}
```

`yinele(4) { … }` kıvrık parantezin içindeki komutları dört kere çalıştırır. Kısa, okunaklı ve Türkçe bir cümle gibi: “dört kere yinele: ileri git, sağa dön.” İkisinin tuvaldeki hâli şöyle:



İlk çizim. `ileri(100)` tek çizgi bıraktı; kaplumbağa `sağ()` ile doğuya döndü, bekliyor.



Kare. `yinele(4)`'ün işi. Kaplumbağa başladığı köşede, başladığı yöne dönmüş olarak durur — dört dönüş tam 360 derece.

DENEYİN

Karenin kodunda iki sayı var: `4` ve `100`. Bir de `sağ()`'ın gizli sayısı: 90 derece. `yinele(3)` ile `sağ(120)` yazarsanız ne çıkar? `yinele(6)` ile `sağ(60)`? Peki `yinele(5)` ile `sağ(144)`? (Sonuncusu güzel bir sürpriz.)

PÜF NOKTASI

Örneklerimiz `sil()` ile başlamıyor: iKojo her çalıştırmada tuvali kendiliğinden temizliyor, o satıra gerek kalmıyor. Ortamınız çizimi silmiyorsa ya da çizimin ortasında tuvali boşaltmak isterseniz komut elinizin altında — aşağıdaki tabloda.

Kaplumbağanın epey marifeti var; en çok kullanacaklarınız şunlar:

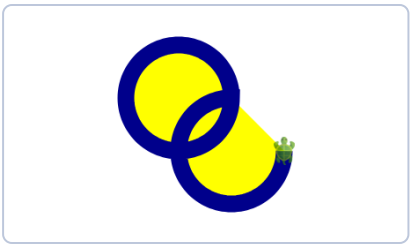
KOMUT	NE YAPAR
`ileri(100)`, `geri(50)`	Baktığı yönde yürür (iz bırakarak)
`sağ(45)`, `sol(30)`	Olduğu yerde döner; sayı vermezseniz 90 derece
`zıpla(50)`	Kalemi kaldırıp yürür: iz bırakmaz
`atla(x, y)`	Çizmeden `(x, y)` noktasına gider
`daire(60)`, `yay(60, 120)`	Yarıçapı verilen daireyi ya da yayı çizer
`kalemRenginiKur(mavi)`	Kalemin rengini değiştirir
`boyamaRenginiKur(sarı)`	Kapalı şekillerin içini boyar
`kalemKalınlığınıKur(4)`	Çizginin kalınlığını ayarlar
`canlandırmaHızınıKur(10)`	Kaplumbağayı hızlandırır (100 adımın süresi, milisaniye)
`sil()`	Tuvali temizler, kaplumbağayı eve gönderir

> **PÜF NOKTASI**
>
> Renkler de hazır: `mavi`, `kırmızı`, `sarı`, `yeşil`, `mor`, `turuncu`, `camgöbeği`... Bir de `rastgeleRenk` var — her çağrıda başka bir renk verir. Deseninizin her koluna bir `kalemRenginiKur(rastgeleRenk)` koyun, ne olacağını görün.

Tablodaki komutların çoğunu tek seferde deneyen yedi satırlık bir program — ve tuvalde bıraktığı iz:

çeşitli komutlar

```
hızıKur(orta)
kalemRenginiKur(Renkler.koyuMavi)
boyamaRenginiKur(sarı)
kalemKalınlığınıKur(19)
daire(60)
sol()
yay(60, 270)
```



Daire ve yay. Kalın kalem, dolgu rengi, tam daire ve 270 derecelik yay — hepsi bir arada. `hızıKur(orta)` da `canlandırmaHızınıKur`'un sözle söyleneni: `yavaş`, `orta`, `hızlı`, `çokHızlı` olur.

I.2

Kendi komutunuz: `tanım`

Kare çizen kodu her seferinde yeniden yazmak olmaz. Ona bir **ad** verelim ki tek sözcükle çağırabilelim. İşte ilk çekirdek sözcüğümüz: `tanım`. (Scala'daki özgün adı `def`; Koco ikisini de anlar, biz Türkçesini kullanacağız.)

kare.kojo — İLK İŞLEVİNİZ

```
tanım kare(kenar: Sayı) = {
  yinele(4) {
    ileri(kenar)
    sağ()
  }
}

kare(50)
kare(100)
kare(150)      // iç içe üç kare
```

`kare` artık bir **komut**: girdisi var (`kenar`), işi var (dört kenar çizmek). Girdinin yanındaki `: Sayı` ise girdinin **türü** — kenar bir tam sayı olmalı, yazı ya da renk değil. Türlere birazdan döneceğiz.

Şimdi asıl güzellik. Madem `kare(100)` tek komut, onu da yineleyebiliriz:

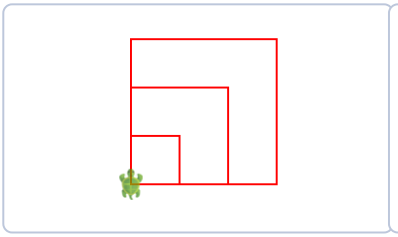
döner desen

```
canlandırmaHızınıKur(10)
yinele(12) {
  kare(100)
  sağ(30)     // 12 × 30 = 360: tam tur
}
```

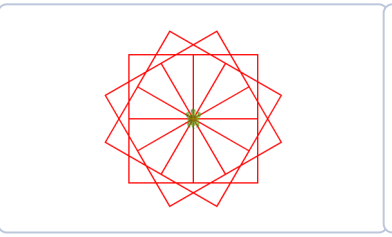
On iki kare, her biri 30 derece dönük: bir çiçek. Dört satırlık kodun böyle bir şey çizmesi ilk seferinde insanı hep şaşırtıyor. Kaç kere döneceğinizi bilmek isterseniz sayaçlı yineleme de var:

```
sayaçlı yineleme

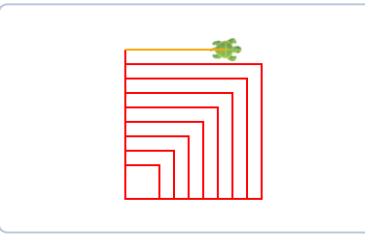
yineleDizinli(10) { k =>      // k: 0, 1, 2, ... 9
  kare(20 + 15 * k)           // her kare bir öncekinden büyük
}
```



kare.kojo. Aynı işlev, üç boy: `kare(50)`, `kare(100)`, `kare(150)`.



Döner desen. On iki kare, 30'ar derece: çiçek.



Sayaçlı yineleme. `k` büyüdükçe kare büyüyor: 20, 35, 50…

TAKIM İŞİ

Biriniz kâğıda bir desen çizsin — makul olsun: iç içe üçgenler, bir merdiven, bir yel değirmeni. Öteki onu koda çevirsin. Sonra roller değişsin. Kâğıttaki şekli koda çevirirken sorduğunuz sorular (“şimdi kaç derece dönmeli?”) tam olarak programcı sorularıdır.

I.3

Değerler ve türler

İkinci çekirdek sözcük: `dez`, yani **değişmez değer** (Scala'daki adı `val`). Bir şeye ad takar ve o ad artık hep o değeri gösterir:

```
değerler

dez kenar = 80
dez boşluk = kenar / 4
dez ad = "kelebek"

satıryaz(ad)                // çıktı penceresine yazar
satıryaz(kenar + boşluk)    // 100
satıryaz(s"kenar $kenar, boşluk $boşluk")  // yazı kalıbı
```

Son satırdaki `s"…"` bir *yazı kalıbı*: içindeki `$kenar` yazının ortasına değerin kendisini koyar. Çıktıya bir şey yazdırmanın en tatlı yolu budur.

Dikkat ederseniz `dez kenar = 80` yazarken tür söylemedik. Scala türü kendisi *çıkarır*: 80 bir `Sayı`, `"kelebek"` bir `Yazı`. Tür yine var — sadece çoğu zaman yazmak zorunda değilsiniz. En çok göreceğiniz türler:

TÜR	NE TUTAR	ÖRNEK
`Sayı`	Tam sayı	`16`, `-3`
`Kesir`	Kesirli sayı	`1.72`
`Yazı`	Harf dizisi	`"merhaba"`
`Harf`	Tek harf	`'B'`
`İkil`	`doğru` ya da `yanlış`	`5 > 3`
`Uzun`, `İriSayı`	Büyük tam sayılar	birazdan…

Her türün bir sınırı var — burada da

Kardeş kitapçıkta C++'ın `int` türünün taşmasını görmüştük. Kötü haber: `Sayı` da aynı 32 bitlik kutuda yaşıyor. Tahmin edin, sonra çalıştırın:

taşma — TANIDIK BİR HATA

```
dez a = 2000
satıryaz(a * a * a)          // 2000'in küpü... öyle mi?

dez iri: İriSayı = 2000      // istediği kadar büyüyebilen tür
satıryaz(iri * iri * iri)
```

çıktı

```
-589934592
8000000000
```

Evet: eksi! Sayaç doldu ve başa döndü — C++'taki taşmanın (*overflow*) ta kendisi, aynı sayıyla, aynı sonuçla. Diller değişir, bilgisayarın belleği değişmez. `İriSayı` ise dilediği kadar basamak kullanır; yarışmalardaki “büyük sayı” sorularının Scala'daki kestirmesi budur.

Ya değişmesi gerekiyorsa: `den`

`dez` ile tanımlanan bir ada yeni değer veremezsiniz; denerseniz Koco daha çalıştırmadan itiraz eder. Bu bir eksiklik değil, bu kitapçığın en önemli fikirlerinden biri: **değişmeyen değere güvenilir.** Kimin ne zaman değiştirdiğini düşünmek zorunda kalmazsınız, çünkü kimse değiştiremez.

Yine de arada gerçekten değişen bir şey gerekir; onun için de `den` var: **değişken** (Scala'daki adı `var`). Kural şu: **önce `dez` deneyin; `den`'e ancak mecbur kalınca dönün.** Bu bölümün sonunda bir kere mecbur kalacağız.

Çıktısı olan işlevler

`kare` bir şeyler *yapıyordu*. İşlevlerin asıl gücü ise bir şeyler *hesaplamak*: girdi al, çıktı ver.

hesaplayan işlevler

```
tanım topla(a: Sayı, b: Sayı) = a + b

tanım bölünüyorMu(sayı: Sayı, bölen: Sayı) = sayı % bölen == 0

satıryaz(topla(3, 5))            // 8
satıryaz(bölünüyorMu(15, 5))     // doğru
satıryaz(bölünüyorMu(16, 5))     // yanlış
```

`return` gibi bir sözcük aramayın: Scala'da işlevin gövdesi bir *deyiş*tir ve değeri neyse çıktı odur. `a + b` yazdınız, çıktı `a + b`. Kısa işlevler tek satıra sığar ve matematikteki tanımlar gibi okunur — işlevsel üslubun ilk tadı.

Karar vermek için üçüncü ve dördüncü çekirdek sözcükler: `eğer` ve `yoksa` (*if/else*). Bunlar da deyiş; yani bir *değer üretirler*:

eğer bir deyiştir

```
tanım enİri2(a: Sayı, b: Sayı) = eğer (a > b) a yoksa b

tanım işaret(s: Sayı) =
  eğer (s > 0) "artı"
  yoksa eğer (s < 0) "eksi"
  yoksa "sıfır"

satıryaz(enİri2(7, 3))     // 7
satıryaz(işaret(-5))       // eksi
```

SIK YANILGI

- `=` **ile** `==`**.** Tek eşittir ad verir ya da değer atar, çift eşittir karşılaştırır. `eğer (a = b)` yazarsanız Koco çalıştırmadan itiraz eder; kardeş dilde aynı satır sessizce geçer. Bu kitapçığın bir yararı da bu: derleyici çok şeyi daha baştan yakalar.
- `7 / 2` **üç eder.** İki `Sayı`'nın bölümü yine `Sayı`; kesir atılır. `3.5` isterseniz taraflardan biri `Kesir` olmalı: `7.0 / 2`. Kalan için `7 % 2`.
- **Bir eksik, bir fazla.** `1 |- 10` dokuz sayı verir (10 hariç), `1 |-| 10` on sayı. Aralık yazarken son sayının içeride mi dışarıda mı kaldığını her seferinde sorun.

İlk Project Euler çözümünüz

Project Euler'in birinci sorusu: 1000'den küçük, 3'e ya da 5'e tam bölünen bütün sayıların toplamı kaç? Kardeş kitapçıkta bunu bir döngüyle çözmüştük. İşlevsel üslupta döngü bile yok:

pe1.kojo — TEK DEYİŞLİK ÇÖZÜM

```
dez toplam = (1 |- 1000)               // 1'den 1000'e kadar (1000 hariç)
  .ele(s => bölünüyorMu(s, 3) || bölünüyorMu(s, 5))   // uyanları ele: süz
  .topla                               // hepsini topla

satıryaz(toplam)     // 233168
```

Satır satır okuyun: *1'den 1000'e kadar sayılardan, 3'e ya da 5'e bölünenleri ele; topla.* Kod, sorunun Türkçesiyle neredeyse birebir. `ele` ve arkadaşlarını II. Bölüm'de derinlemesine göreceğiz; şimdilik tanışmış olun. (`||` "ya da" demek, `&&` de "ve" — C++'taki `or` / `and` sözcüklerinin imgeli halleri.)

> **KLASİK TUZAK**
>
> Bu soruyu "3'e bölünenler + 5'e bölünenler" diye iki ayrı toplamla çözerseniz 15'e bölünenleri iki kere saymış olursunuz. Kardeş sınıfta bu hatayı gerçekten yaptık. Yukarıdaki `ele`'li çözümde bu tuzak *kurulamıyor* bile — her sayı bir kere elenir ya da kalır. İyi araç, bazı hataları yazılamaz hâle getirir.

I.5

İlk oyununuz

Bilgisayar 1 ile 100 arasında bir sayı tutsun, siz bulmaya çalışın. Kaplumbağa bu sefer dinlensin; oyun küçük bir soru penceresinde geçiyor:

tahmin.kojo — SAYI TAHMİN OYUNU

```
dez tutulan = rastgele(100) + 1   // 1..100 -- "gizli" diyemezdik: o anahtar sözcüklerden biri!
den hak = 7                       // işte o mecbur kaldığımız den!
den bulundu = yanlış

den msj = s"1 ile 100 arasında bir sayı tuttum. $hak hakkın var."

yineleDoğruysa(hak > 0 && !bulundu) {
  dez tahmin = sayıOku(msj + " Tahminin?")
  hak = hak - 1
  eğer (tahmin == tutulan) {
    bulundu = doğru
    satıryaz(s"Bildin! $hak hakkın kalmıştı.")
  } yoksa {
    dez ne = eğer (tahmin < tutulan) "büyük" yoksa "küçük"
    msj = s"Daha $ne olmalı. $hak hakkın kaldı."
  }
}

eğer (!bulundu) satıryaz(s"Hakkın bitti. Tuttuğum sayı $tutulan idi.")
```

Oyun hazır olarak da duruyor: tek satır yazmadan iKojo'da açıp **Çalıştır**'a basabilirsiniz.

`yineleDoğruysa`, koşul doğru kaldıkça yineler — C++'taki `while`'ın Koco'daki karşılığı. `sayıOku` sizden bir sayı ister, `rastgele(100)` 0 ile 99 arasında bir sayı verir. Ve evet, üç tane `den` kullandık: oyunun *durumu* (kalan hak, bulundu mu, sıradaki ileti) gerçekten değişiyor. Değişen şeye değişken demek günah değil; alışkanlık hâline getirmek günah.

> **KÜÇÜK BİR TASARIM KARARI**
>
> İpuçlarını neden `satıryaz` ile yazdırmadık da `msj`'de biriktirip `sayıOku`'nun sorusuna koyduk? Çünkü soru penceresi açıkken altındaki çıktı penceresini kapatıyor; yazdırdığınız ipucunu göremiyorsunuz. İletiyi sorunun *içine* koyunca ipucu hep gözünüzün önünde oluyor. Arayüzün bir kısıtı kodun tasarımını değiştirdi — programcılıkta bu çok olur.

Bir de şu satıra bakın: `dez ne = eğer (tahmin < tutulan) "büyük" yoksa "küçük"`. Az önce "`eğer` bir deyiştir" demiştik; işte tam da o: koşul bir *değer* üretiyor, biz de o değere ad takıyoruz. Son bir ayrıntı: tutulan sayıya `gizli` diyemedik, çünkü `gizli` Koco'da bir anahtar sözcük (Scala'daki `private`). Anahtar sözcükler ad olarak kullanılamaz — C++'ta `if` adında değişken olmaması gibi.

Neden tam yedi hak?

Akıllı oyuncu her tahminde ortayı söyler: önce 50, sonra kalan yarının ortası, sonra onun ortası... Her tahmin arama uzayını ikiye böler ve $2^7 = 128 > 100$ olduğu için yedi tahmin her zaman yeter. Bu yöntemin adı **ikili arama** (*binary search*) ve kardeş kitapçığın da bu kitapçığın da baş köşesinde oturuyor: milyon sayı içinden aradığınızı yirmi soruda bulur.

DENEYİN

Sınırı 1000 yapın. Kaç hak vermek gerekir? Programı, hak sayısını sınırdan kendisi hesaplayacak biçimde değiştirin. (İpucu: `log2tabanlı(1000)` diye hazır bir işlev var; ama önce `yineleDoğruysa` ile elle hesaplamayı deneyin — daha öğretici.)

ALIŞTIRMALAR

1. **Yıldızlı gökyüzü.** Beş köşeli bir yıldız çizen `yıldız(boy: Sayı)` işlevi yazın (I.1'deki sürprizi hatırlayın). Sonra `atla` ve `rastgele` ile tuvalin her yerine, rastgele boylarda yirmi yıldız serpiştirin.
2. **Çember çiçeği.** `daire` ve döndürmeyle, I.2'deki kare çiçeğinin çemberli sürümünü çizin. Renkleri `rastgeleRenk`'ten alın.
3. **Fibonaççi.** Project Euler 2: dört milyonu aşmayan çift Fibonaççi sayılarının toplamı. Şimdilik `den` ve `yineleDoğruysa` ile yazın; Bölüm IV'te bir daha, bambaşka yazacağız.
4. **Büyük çarpan.** Project Euler 3: `600851475143` sayısının en büyük asal çarpanı. TUZAK Bu sayı `Sayı`'ya sığmaz. Hangi türü seçeceksiniz?
5. **Tahmini tersine çevirin.** Siz tutun, bilgisayar bulsun: program her turda ortayı söylesin, siz `b` / `k` / `=` girin. Kaç adımda buluyor? Yukarıdaki hesapla uyuşuyor mu?

TAKIM İŞİ

Üçüncü ve dördüncü alıştırmayı ikili programlamayla yapın: on beş dakika biri yazsın, on beş dakika öteki. Bitince kodu kapatın ve sıfırdan, ezberden bir daha yazın. İkinci yazışın hızına şaşıracaksınız; öğrenme dediğimiz şey tam orada oluyor.

BÖLÜM II

Veriyi Düzenlemek

Tek sayıyla, tek yazıyla ancak bu kadar oynanır; programların gücü çok veriyi düzenli tutmaktan gelir. Bu bölümde Koco'nun kaplarını tanıyacaksınız — dizin, küme, eşlem — ve onların üstünde çalışan üç sihirli işlemi: **işle**, **ele**, **indirge**. Bu üçünü kavrayan, döngülerin yarısını bir daha hiç yazmaz.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

II.1

Dizin: işlevselin atardamarı

İşlevsel dünyanın en sevdiği kap **dizin** (*list*). Ögeleri art arda dizer ve — dikkat — **bir kere kuruldu mu bir daha değişmez**:

dizin

```
dez sayılar = Dizin(3, 1, 4, 1, 5, 9)

satıryaz(sayılar.başı)      // 3  (ilk öge)
satıryaz(sayılar.kuyruğu)   // Dizin(1, 4, 1, 5, 9)  (gerisi)
satıryaz(sayılar.boyu)      // 6
satıryaz(sayılar.tersi)     // Dizin(9, 5, 1, 4, 1, 3)
satıryaz(sayılar.sıralı)    // Dizin(1, 1, 3, 4, 5, 9)
satıryaz(sayılar)           // Dizin(3, 1, 4, 1, 5, 9) -- hâlâ aynı!
```

Son satır bu bölümün ilk büyük fikri. `tersi` ve `sıralı`, `sayılar`'ı değiştirmedi; **yeni birer dizin** verdi. Değiştirilemeyen veriyle çalışmak ilk başta kısıtlayıcı gelir, sonra özgürleştirir: hiçbir işlev, elinizdeki veriyi arkanızdan bozamaz.

> **SIK YANILGI**
>
> `sayılar.sıralı` **yazıp sonucunu almamak.** Değişmez dizinde "sırala" diye bir komut yok; `sıralı` sıralanmış *yeni* bir dizin verir ve siz onu bir ada bağlamazsanız kaybolur. `dez sıralı = sayılar.sıralı` deyin, sonra onunla çalışın. Kardeş dilden gelenler bu tuzağa ilk hafta en az bir kere düşer.

Baş ve kuyruk ikilisinin bir de tersi var: öne öge *eklemek*. Buna “cons” deniyor, imgesi `::`

öne ekleme

```
dez d1 = Dizin(2, 3)
dez d2 = 1 :: d1        // Dizin(1, 2, 3) -- d1 yine değişmedi
dez d3 = 0 :: d2        // Dizin(0, 1, 2, 3)

satıryaz(Boş)           // Dizin() -- boş dizin de bir değer!
satıryaz(7 :: Boş)      // Dizin(7)
```

Her dizin aslında bu ikiliden ibaret: ya `Boş` tur ya da “bir baş ve gerisi”. Bu basit gözlem, Bölüm IV'te özyinelemeyle birleşince çok iş görecek.

II.2

Üç sihirli işlem

Şimdi bu kitapçığın kalbi. Elinizde bir dizin var; onunla yapmak isteyeceğiniz hemen her şey şu üç işlemin bileşimi:

işle *(map)*

Her ögeye aynı işlevi uygula, sonuçların dizinini ver. Boy değişmez, ögeler dönüşür.

ele *(filter)*

Her ögeye bir soru sor; yanıtı `doğru` olanları tut. Ögeler değişmez, boy kısalır.

indirge *(reduce)*

Ögeleri ikişer ikişer birleştirerek tek değere in: toplam, çarpım, en büyük...

üçlü.kojo — İŞLE, ELE, İNDİRGE

```
dez sayılar = Dizin(3, 1, 4, 1, 5, 9, 2, 6)

satıryaz(sayılar.işle(s => s * s))       // Dizin(9, 1, 16, 1, 25, 81, 4, 36)
satıryaz(sayılar.ele(s => s % 2 == 0))   // Dizin(4, 2, 6)
satıryaz(sayılar.indirge((a, b) => a + b))  // 31

// en sık kullanılanların kısaltmaları da hazır:
satıryaz(sayılar.topla)      // 31
satıryaz(sayılar.çarp)       // 6480
satıryaz(sayılar.enİrisi)    // 9
satıryaz(sayılar.enUfağı)    // 1
```

`s => s * s` yazımına dikkat: bu, **adsız bir işlev** (*lambda*). "`s`'yi al, `s * s`'yi ver" diyor. İşlevleri böyle, değermiş gibi elden ele vermek işlevsel programlamanın tam kalbidir: `işle`'ye bir işlev *veriyorsunuz*.

Ve asıl güç, zincirlemede. Bölüm I'deki Project Euler çözümünü hatırlayın; şimdi bir yenisi: *100'e kadar sayıların karelerinden, 3'e bölünenlerin toplamı.*

zincir

```
(1 |-| 100)                    // 1'den 100'e (100 dahil)
  .işle(s => s * s)            // karelerini al
  .ele(k => k % 3 == 0)        // 3'e bölünenleri tut
  .topla                       // ve topla
```

PÜF NOKTASI

`1 |-| 100` uçları dahil bir aralık, `1 |- 100` ise son sayı hariç. Zinciri her adımında `satıryaz` ile kesip ara sonucu yazdırabilirsiniz — işlevsel kodda hata ayıklamanın en kolay yolu, zinciri kısaltıp bakmaktır.

TAKIM İŞİ

Biriniz bir görev söylesin ("çift sayıların karelerinin en büyüğü" gibi), öteki onu `işle` / `ele` / `indirge` zinciriyle yazsın; sonra değişin. On dakikada bir düzine tur atarsınız. Amaç şu refleksi kazanmak: *döngü yazmadan önce bir dur, bu bir zincir mi?*

II.3

Küme ve eşlem

Küme (*set*) matematikten tanıdığınız kümenin ta kendisi: tekrarsız, sırasız. **Eşlem** (*map*) ise bir şeyi başka bir şeye bağlar: şehirden plakaya, harften sayıya.

küme

```
dez k = Küme(3, 1, 4, 1, 5)   // 1 iki kere girdi...
satıryaz(k)                   // Küme(3, 1, 4, 5) -- ...bir kere kaldı
satıryaz(k.içeriyorMu(4))     // doğru
satıryaz(k + 9)               // yeni küme; k değişmedi
```

Kümenin işlevsel dünyada tatlı bir sırrı var: **küme aynı zamanda bir işlevdir.** `k(4)` yazmak "4 kümede mi?" diye sormaktır. Bu küçük ayrıntı Bölüm V'te çizge gezerken kodumuzu güzelleştirecek.

Harf saymak, döngüsüz

Kardeş kitapçıkta bir yazıdaki harfleri `map<char,int>` ve bir döngüyle saymıştık. Burada iş bölümünü `öbekle` (*group by*) yapıyor:

harfsay.kojo — ÖBEKLE

```
dez harfler = "kelebek kanadi".ele(h => h != ' ')

dez öbekler = harfler.öbekle(h => h)
// 'k' -> "kkk", 'e' -> "eee", 'l' -> "l", ...

dez sayaç = öbekler.işle { durum (harf, öbek) => (harf, öbek.boyu) }

sayaç.dizine.sıralı.herbiriİçin { durum (harf, kaç) =>
  satıryaz(s"$harf -> $kaç")
}
```

çıktı

```
a -> 2
b -> 1
d -> 1
e -> 3
i -> 1
k -> 3
l -> 1
n -> 1
```

Üç yenilik birden girdi, teker teker: `öbekle` ögeleri verdiğiniz işleve göre öbeklere ayırır ve bir eşlem verir. `(harf, öbek)` bir **demet** (*tuple*): iki değeri yan yana taşıyan en sade paket. `durum (harf, kaç) =>` ise demeti yakalayıp parçalarına ad veren **eşleştirme** (*pattern matching*) — yeni çekirdek sözcüğümüz `durum` (*case*) işte bu.

Eşlem ve `Belki`

plaka.kojo — EŞLEM

```
dez plaka = Eşlem("Adana" -> 1, "Ankara" -> 6, "İstanbul" -> 34)

satıryaz(plaka("Ankara"))          // 6
satıryaz(plaka.al("Ankara"))       // Some(6)   -- Koco'daki adı: Biri(6)
satıryaz(plaka.al("Mars"))         // None      -- Koco'daki adı: Hiçbiri
satıryaz(plaka.alYoksa("Mars", 0)) // 0

plaka.eşEkle("İzmir" -> 35)        // bu eşlem büyüyebilen cinsten
satıryaz(plaka.sayı)               // 4
```

Buradaki incelik `al`'ın çıktısı. "Mars'ın plakası kaç?" sorusunun dürüst yanıtı bir sayı değil; *"yok öyle bir şey"*. İşlevsel diller bu yanıtı da bir değere çevirir: `Belki` (*Option*) türü ya `Biri(6)` dır ya `Hiçbiri`. Kardeş kitapçıkta eşlemde olmayan anahtarı `[]` ile okumanın anahtarı *sessizce*

yarattığını, bunun yarım saatimizi yediğini anlatmıştım. `Belki` tam da o tuzağın panzehiri: derleyici, "ya yoksa?" durumunu ele almayı size *hatırlatır*.

> **DİKKAT**
>
> `plaka("Mars")` yazarsanız program o anda hata verip durur. Emin değilseniz `al` ya da `alYoksa` kullanın. "Çalışırken patlayan" ile "derlenmeden yakalanan" hata arasındaki fark, bu kitapçığın en çok döneceği ayrımlardan.

II.4

Sırası önemli kaplar

Kardeş kitapçıktan tanıdığınız üçlü burada da var; adları Türkçe:

KAP	KURAL	YÖNTEMLER	NE İŞE YARAR
Yığın `Yığın`	Son giren ilk çıkar	`koy` `al` `tepe`	Derinlemesine gezi, geri alma
Kuyruk `Kuyruk`	İlk giren ilk çıkar	`ekle` `baştanAl` `başı`	Enlemesine gezi, sıra bekleyen işler
Öncelik sırası `ÖncelikSırası`	En öncelikli önce çıkar	`ekle` `baştanAl`	Dijkstra, "en yakın n şey"

yığın ve kuyruk

```
dez yığın = Yığın[Sayı]()
dez kuyruk = Kuyruk[Sayı]()

Dizin(3, 1, 4).herbiriİçin { s =>
  yığın.koy(s)
  kuyruk.ekle(s)
}

satıryaz(yığın.al())      // 4 -- son giren
satıryaz(kuyruk.baştanAl())  // 3 -- ilk giren
```

Bu ikisi *değişen* kaplar; işlevsel saflıktan taviz veriyorlar ama dert değil — doğru işte doğru araç. Bölüm V'te göreceksiniz: aynı gezi kodunda yığını kuyrukla değiştirmek, algoritmayı derinlemesineden enlemesineye çevirecek.

ALIŞTIRMALAR

1. **Sesli harfler.** Bir yazıdaki sesli harfleri `say` ile sayın: `"aeıioöuü"` kümesini kurun ve kümenin işlev olduğunu kullanın. Tek satır yeter.
2. **Tekrar var mı?** Bir dizinde tekrar eden öge var mı? İpucu: `yinelemesiz` ve `boyu`. Tek satır. Sonra aynı soruyu kümeye çevirerek çözün ve iki çözümü karşılaştırın.
3. **En uzun sözcük.** Bir sözcük dizininden en uzununu bulun. İpucu: `enİrisi`'nin işlev alan hâli var: `enİrisi(s => s.boyu)`.
4. **Kelime sayacı.** Harf sayacını kelime sayacına çevirin: bir cümledeki her kelime kaç kere geçiyor? İpucu: yazıyı boşluklardan bölmek için `böl(' ')` hazır — bir kelime dizini verir, gerisi harf sayacıyla aynı.
5. **Ters sözlük.** Plaka eşlemini tersine çevirin: sayıdan şehre. `işle` ve demetlerle tek zincirde olur. `DÜŞÜNDÜRÜCÜ` İki şehir aynı plakayı paylaşsaydı ne olurdu? Deneyin ve açıklayın.

BÖLÜM III

Soyutlama Sanatı

Program yazmak üç becerinin bileşimi: yapıtaşlarını tanımak, onları birleştirmek ve — en önemlisi — bileşimlere ad takıp yeni yapıtaşları üretmek. Bu sonuncusunun adı **soyutlama**. Bu bölümde tekrarları ada çevireceğiz, işlevleri işlevlere vereceğiz, kendi `yinele`'mizi yazacağız — ve programlarımızı kendi kendini sınar hâle getireceğiz.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

III.1

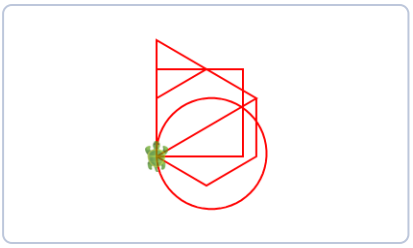
Tekrara ad takın

Bölüm I'de üçgen, kare ve altıgen çizdiniz. Üç ayrı işlev yazdıysanız, şimdi yan yana koyup bakın: aralarındaki tek fark bir sayı. Yazılımın en meşhur ilkesi burada devreye girer: **kendini tekrar etme** (*DRY — Don't Repeat Yourself*). Tekrarı görünce ona bir ad takın:

çokgen.kojo — ÜÇ İŞLEV → BİR İŞLEV

```
tanım çokgen(kenarSayısı: Sayı, kenar: Kesir) = {
  yinele(kenarSayısı) {
    ileri(kenar)
    sağ(360.0 / kenarSayısı)     // dış açıların toplamı hep 360
  }
}

çokgen(3, 120)     // üçgen
çokgen(4, 90)      // kare
çokgen(6, 60)      // altıgen
çokgen(60, 6)      // 60 kenarlı çokgen... neredeyse çember!
```



Tek işlev, dört çokgen. Üçgen, kare, altıgen ve 60 kenarlı "çember" — hepsi aynı `çokgen`'den.

Son satır güzel bir armağan: kenar sayısını artırınca çokgen çembere yaklaşıyor. Bir soyutlama iyi kurulduysa, beklemediğiniz yerlerde işe yarar — iyi kurulduğunun işareti de budur.

> **PÜF NOKTASI**
>
> Soyutlamanın yarısı **iyi ad koymaktır**. `çokgen(kenarSayısı, kenar)` kendini anlatıyor; `f(n, x)` anlatmıyor. Kodunuzu iki hafta sonra okuyacak en önemli kişi için yazın: iki hafta sonraki siz.

III.2

İşlevi girdi alan işlev

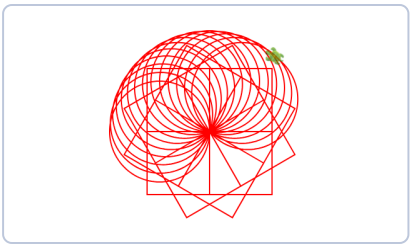
Bölüm II'de `işle`'ye adsız işlevler verdik. Şimdi perdeyi aralayıp aynı gücü kendi işlevlerimize verelim. Şu iki desene bakın: "12 kare, her biri 30 derece dönük" ve "36 daire, her biri 10 derece dönük". Tekrar eden şey bu sefer bir sayı değil, *çizimin kendisi*. O da soyutlanır:

dönence.kojo — ÇİZİMİ GİRDİ YAP

```
tanım dönence(kaç: Sayı, açı: Kesir)(çizim: => Birim) = {
  yinele(kaç) {
    çizim
    sağ(açı)
  }
}

canlandırmaHızınıKur(10)

dönence(12, 30) { çokgen(4, 100) }   // kare çiçeği
dönence(36, 10) { daire(80) }        // halka halka daireler
dönence(8, 45) {                     // blok uzayabilir de:
  kalemRenginiKur(rastgeleRenk)
  çokgen(3, 80)
}
```



Üç dönence bir arada. Kare çiçeği, 36 dairelik halka ve rastgele renkli üçgenler üst üste — dantel gibi.

`çizim: => Birim` girdisi bir kod bloğu: *çalıştırılmamış komutlar*. `dönence` onu istediği kadar çalıştırıyor. Yani işlevler de tıpkı sayılar gibi elden ele verilebilen değerlerdir — işlevsel programlamaya adını veren fikir tam olarak bu. (`Birim`, "kayda değer bir çıktısı yok, işi var" demek; C++'taki `void`'in kardeşi.)

Perde arkası: `yinele` dediğiniz nedir ki?

Baştan beri kullandığımız `yinele` de işte böyle bir işlev. Kendi sürümümüzü yazabiliriz:

bizimYinele

```
tanım bizimYinele(kaç: Sayı)(komutlar: => Birim): Birim =
  eğer (kaç > 0) {
    komutlar
    bizimYinele(kaç - 1)(komutlar)    // kendini çağırıyor!
  }

bizimYinele(4) { ileri(100); sağ() }    // aynı kare
```

İki şeye bakın. Birincisi: dilin "sihirli" sandığınız parçası beş satırlık sıradan bir işlevmiş. İkincisi: `bizimYinele` kendini çağırıyor. Bunun adı **özyineleme** ve bir sonraki bölümün tamamı onun; şimdilik sadece not edin.

> **TAKIM İŞİ**
>
> Biriniz `bizimYineleDizinli(kaç)(komutlar: Sayı => Birim)` işlevini yazsın — hani şu sayacı da veren sürüm. Öteki, onu hiç görmeden sadece çağırarak sınasın: `bizimYineleDizinli(5) { k => satıryaz(k) }` ne yazmalı? Sıfırdan mı başlamalı, birden mi? Bu kararın adı *arayüz tasarımı* ve tartışması yazmasından değerlidir.

III.3

Kendi kendini sınayan program

Bir işlev yazdınız, birkaç değerle denediniz, doğru göründü. Sonra bir yerini değiştirdiniz — hâlâ doğru mu? Bunu her seferinde gözle denetlemek olmaz. Çözüm, denemeleri programın içine koymak: `belirt` (*assert*) verilen koşul yanlışsa programı o an durdurur ve verdiğiniz mesajı gösterir.

asal.kojo — ÖNCE DENEMELER

```
tanım asalMı(n: Sayı): İkil =
  n > 1 && (2 |- n).hepsiDoğruMu(b => n % b != 0)

tanım denemeler() = {   // "dene" diyemezdik: o bir anahtar sözcük (try)
  belirt(asalMı(2), "2 asaldır")
  belirt(asalMı(97), "97 asaldır")
  belirt(!asalMı(1), "1 asal sayılmaz")
  belirt(!asalMı(91), "91 = 7 × 13, kolay tuzak")
  satıryaz("denemeler geçti.")
}

denemeler()
```

`asalMı`'nın gövdesini yüksek sesle okuyun: *n birden büyükse ve 2'den n'ye kadar hiçbir sayı onu bölmüyorsa, asaldır.* Tanımın Türkçesi ile kodun kendisi neredeyse aynı cümle — işlevsel üslubun sınamayı kolaylaştırmasının sebebi de bu: kod tanıma benzeyince tanımla karşılaştırması kolaylaşıyor.

`!asalMı(91)` satırı özellikle önemli. İyi bir deneme kümesi yalnız doğruları değil, **yanlış olması gerekenleri** de sınar. 91 asal *görünür*; asallık kodlarının klasik tuzağıdır. Kardeş sınıfta C++ yazarken de aynı sayıyı denemiştik — diller değişiyor, tuzaklar değişmiyor.

> **DENEYİN**
>
> `asalMı` içindeki `n > 1` koşulunu silin ve `denemeler()`'i çalıştırın. Hangi deneme, hangi mesajla patlıyor? Denemelerin işi tam olarak bu: siz kodu bozar bozmaz, hangi sözünüzü bozduğunuzu söylemek.

III.4

Tahmin etmeyin, ölçün — ve sayın

`asalMı` doğru; peki hızlı mı? `n` için 2'den `n`'ye kadar *bütün* sayıları deniyor. Oysa `n = a × b` ise çarpanlardan biri mutlaka `√n`'den büyük değildir; o hâlde **kareköke kadar bakmak yeter**. Fark ne kadar? Ölçmeden önce bir sayalım:

N	2'DEN N'YE DENEME	√N'YE KADAR DENEME
10 007	10 005	99
1 000 003	1 000 001	999
$10^9 + 7$	~10^9	~31 623

Son satırda fark otuz bin kat. Bu tablo bir ölçüm değil, bir *sayım* — daha kodu yazmadan, kâğıt üstünde belli. Yine de sözümüze sadık kalalım ve ölçelim; Koco'da süre tutmak için `buSaniye` var:

süre tutmak

```
tanım süresi(iş: => Birim): Kesir = {
  dez t0 = buSaniye         // şu anın saniyesi
  iş
  buSaniye - t0
}

satıryaz(süresi { asalMı(999999937) })       // bir süre bekleyeceksiniz...
// asalMıHızlı'yı yazınca onu da ölçün ve karşılaştırın
```

`süresi`'ne dikkat: yine işlevi girdi alan bir işlev — bu bölümün bütün fikirleri birbirine dolanıyor. `asalMıHızlı`'yı ise yazmıyorum; karekök sınırıyla siz yazın, denemelerden geçirin, sonra iki sürümü aynı sayıyla ölçüp farkı kendi gözünüzle görün.

DİKKAT

Ölçtüğünüz süreler bilgisayardan bilgisayara değişir; bu yüzden bu kitapçık size "benim makinemde şu çıktı" diye sayı vermiyor — kendi sayılarınızı üretin. Değişmeyen şey *oran*dır: deneme sayısını otuz bin kat azaltan bir fikir, her makinede kabaca o kadar hızlandırır. **Büyük hızlar donanımdan değil, algoritmadan gelir.**

ALIŞTIRMALAR

1. **Sayaçlı yineleme.** Takım işindeki `bizimYineleDizinli`'yi bitirin. Sonra onunla, her basamağı bir öncekinden uzun bir merdiven çizin.
2. **Yıldıza genelleme.** `çokgen`'i `yıldız(köşe: Sayı, kenar: Kesir)` olacak biçimde genelleyin. Beş köşeli yıldızda dönüş açısı kaçtı? Yedi köşelide? Formülü bulun, koda koyun.
3. **Mükemmel sayılar.** Bir sayının kendisi dışındaki bölenlerinin toplamı kendisine eşitse ona mükemmel denir (6 = 1+2+3). `mükemmelMi(n)` işlevini `ele` ve `topla` ile tek satırda yazın; 10 000'e kadar mükemmel sayıları bulun. Kaç tane var?
4. **asalMıHızlı.** Karekök sınırlı sürümü yazın. `denemeler()`'e iki sürümü karşılaştıran bir satır ekleyin:
   `belirt((1 |-| 1000).hepsiDoğruMu(n => asalMı(n) == asalMıHızlı(n)), "iki sür`
   Bu tür denemeye *eşleme denemesi* denir: yavaş ama basit sürüm, hızlı sürümün hakemidir.
5. **Ölçün.** Bölüm II'deki "tekrar var mı" alıştırmasının iki çözümünü `süresi` ile on bin, yüz bin ve bir milyon ögelik dizinlerde ölçün. Süreler nasıl büyüyor? Bir tablo yapın. ÖNEMLİ
   Bu tablo, kardeş kitapçığın IV. bölümündeki *karmaşıklık* konusunun kapısıdır.

BÖLÜM IV

Özyineleme ve Fraktallar

Bir işlev kendi kendini çağırabilir. Kardeş kitapçıkta bu fikri sayılarla ve kulelerle ehlileştirmiştik; burada bir üstünlüğümüz var: özyinelemeyi *çizdireceğiz*. Sarmallar, ağaçlar ve bir kar tanesi — kendini çağıran işlevin neye benzediğini gözünüzle göreceksiniz. Sonra da onu unutkanlıktan kurtarıp kırk bin kat hızlandıracağız.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama
IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

IV.1

Taban ve adım

Bölüm III'ün sonunda `bizimYinele` kendini çağırmıştı. Genel kural şu: her özyinelemeli işlevin iki parçası vardır ve **ikisi de olmazsa olmaz**:

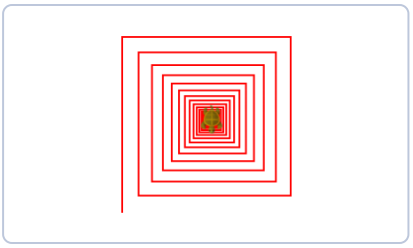
1. **Taban durumu.** Kendini çağırmadan durduğu, en küçük hâl.
2. **Adım.** Problemi kendinden *daha küçük* bir problemle ifade etmesi. Küçülmek şart; küçülmeyen özyineleme hiç durmaz.

İlk örneğimiz çizsin de görelim: her turda biraz kısalan bir sarmal.

sarmal.kojo

```
tanım sarmal(boy: Kesir): Birim =
  eğer (boy > 2) {          // TABAN: boy 2'ye düşünce dur
    ileri(boy)
    sağ()
    sarmal(boy * 0.95)      // ADIM: biraz küçüğünü çiz
  }

canlandırmaHızınıKur(10)
sarmal(200)
```



Sarmal. Her adım bir öncekinin yüzde 95'i; kaplumbağa ortada, taban durumunda durmuş.

Tuvalde içe içe kıvrılan bir kare sarmalı belirecek. Şimdi kodda iki küçük deney yapın: taban koşulunu `boy > 150` yapın — sarmal üç-beş çizgide bitiyor. Sonra `boy * 0.95`'i `boy` yapın; yani küçülmeyi kaldırın. Koco bir süre sonra programı durduracaktır: küçülmeyen özyineleme *hiç durmaz*, buna çarpılmadan öğrenmenin en ucuz yolu da bir çizim programıdır.

> **BİR NUMARALI HATA**
>
> Özyinelemede hataların şahı, taban durumunu unutmak ya da yanlış yazmaktır. Alışkanlık edinin: özyinelemeli bir işlev yazarken **ilk yazdığınız satır taban olsun**. Kardeş kitapçık da aynı cümleyi kurar; iki dilde de doğru olan cümleler en değerlileridir.

IV.2

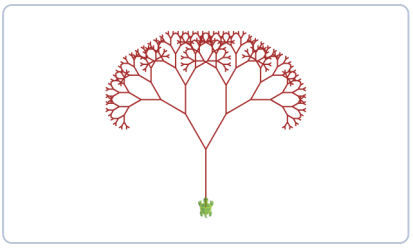
Bir ağaç çizelim

Sarmalda işlev kendini *bir* kere çağırdı. Ya iki kere çağırırsa? Gövdeden iki dal, her daldan iki dal daha... Bu bir ağaç:

ağaç.kojo — İKİ DALLI ÖZYİNELEME

```
tanım ağaç(boy: Kesir): Birim =
  eğer (boy > 4) {
    ileri(boy)              // gövdeyi çiz
    sol(30)
    ağaç(boy * 0.7)         // sol dal: küçük bir ağaç!
    sağ(60)
    ağaç(boy * 0.7)         // sağ dal: bir küçük ağaç daha
    sol(30)
    geri(boy)               // GERİ DÖN: geldiğin gibi bırak
  }

canlandırmaHızınıKur(1)
kalemRenginiKur(kahverengi)
ağaç(90)
```



Ağaç. On üç satırın çizdiği. Her dal, ucunda iki küçük ağaç taşıyor — tanımın ta kendisi.

Tanım bir cümle: *ağaç, ucunda iki küçük ağaç taşıyan bir gövdedir.* Saçma gibi duran bu cümle, ekranda gerçek bir ağaca dönüşüyor. Son satırdaki `geri(boy)` hayati: her dal, kaplumbağayı **bulduğu yere geri bırakıyor** ki kardeş dal doğru yerden başlasın. Kardeş kitapçıktaki geri dönüşlü aramanın "bıraktığın gibi bırak" kuralı, işte bu `geri`.

> **DENEYİN**
>
> Açıları (30 ve 60), oranı (0.7) ve tabanı (4) değiştirin: söğüt, çam, bodur çalı... hepsi bu üç sayının içinde saklı. Bir de her dalda kalem kalınlığını `boy / 10` yapmayı deneyin — gövde kalın, uçlar ince: bir anda gerçekçi oluverir.

IV.3

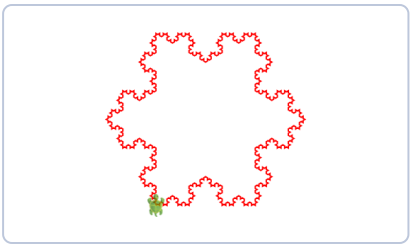
Koch tanesi

Kapaktaki kar tanesine geldik. Kural tek cümle: *bir doğru parçası al, ortadaki üçte birini kes, yerine iki kenarlı bir çatı koy* — sonra aynı şeyi yeni parçaların her birine yap.

koh.kojo — KAR TANESİ

```
tanım koh(boy: Kesir, derinlik: Sayı): Birim =
  eğer (derinlik == 0)
    ileri(boy)                // TABAN: düz çizgi
  yoksa {
    koh(boy / 3, derinlik - 1); sol(60)
    koh(boy / 3, derinlik - 1); sağ(120)
    koh(boy / 3, derinlik - 1); sol(60)
    koh(boy / 3, derinlik - 1)
  }

canlandırmaHızınıKur(0)
yinele(3) {           // üç kenarlı "üçgen", her kenarı bir koh eğrisi
  koh(243, 4)
  sağ(120)
}
```



Koch tanesi. `derinlik = 4`: üç koh eğrisi, 120'şer derece dönerek kapanıyor. Kapaktaki tanenin ta kendisi.

`derinlik` girdisine dikkat: bu sefer küçülen şey uzunluk değil, **derinliğin kendisi**. Sıfırda düz çizgi çiziyoruz; her derinlik katı, çizgiyi dört parçalı bir kıvrıma çeviriyor. `derinlik = 0, 1, 2, 3, 4` için ayrı ayrı çalıştırın ve şeklin nasıl "pürüzlendiğini" izleyin.

Böyle şekillere **fraktal** deniyor: neresine yaklaşırsanız yaklaşın aynı deseni görürsünüz, çünkü tanımı kendine yapılmış bir göndermeden ibaret. Özyinelemeli kod ile fraktal, aynı fikrin biri yazıyla biri çizgiyle hâli.

> **BİR SAYALIM**
>
> Her derinlik katı çizgi sayısını dörde katlar: derinlik 4'te `3 × 4⁴ = 768` çizgi. Derinlik 10'da üç milyondan fazla. Özyinelemenin büyüme hızıyla ilk tanışmanız bu olsun; Hanoi'de sayılar daha da çılgınlaşacak.

IV.4

Hanoi kuleleri

Çizimden sayıya dönelim; klasiklerin klasiği. Üç çubuk, birincisinde büyükten küçüğe n disk; hepsi üçüncü çubuğa taşınacak. Her seferinde tek disk, büyük disk küçüğün üstüne asla.

hanoi.kojo — ÜÇ SATIRLIK ÇÖZÜM

```
den adım = 0

tanım taşı(n: Sayı, kaynak: Sayı, hedef: Sayı, yardımcı: Sayı): Birim =
  eğer (n > 0) {                       // taban: taşınacak disk kalmadı
    taşı(n - 1, kaynak, yardımcı, hedef)   // üstteki n-1 diski kenara çek
    adım = adım + 1
    satıryaz(s"$adım) $n. diski $kaynak -> $hedef")
    taşı(n - 1, yardımcı, hedef, kaynak)   // n-1 diski üstüne koy
  }

taşı(3, 1, 3, 2)
```

çıktı — N = 3

```
1) 1. diski 1 -> 3
2) 2. diski 1 -> 2
3) 1. diski 3 -> 2
4) 3. diski 1 -> 3
5) 1. diski 2 -> 1
6) 2. diski 2 -> 3
7) 1. diski 1 -> 3
```

Yedi adım; genel olarak `2ⁿ − 1`. Kardeş kitapçıkta anlattığım efsaneyi burada da analım: 64 diskli kulenin bitişi, saniyede bir hamleyle yaklaşık **584 milyar yıl** sürer. Rahat olun, kimse beklemiyor.

Dizinlerde özyineleme

Bölüm II'de demiştik: her dizin ya `Boş`tur ya da "baş ve gerisi". Bu, özyineleme için biçilmiş kaftan — ve `eşle` / `durum` (*match/case*) eşleştirmesiyle birleşince tanımlar şiire dönüyor:

dizin özyinelemesi — KENDİ TOPLA'MIZ

```
tanım toplam(d: Dizin[Sayı]): Sayı = d eşle {
  durum Boş          => 0                    // TABAN: boş dizinin toplamı
  durum baş :: gerisi => baş + toplam(gerisi)  // ADIM
}

tanım altKümeler(d: Dizin[Sayı]): Dizin[Dizin[Sayı]] = d eşle {
  durum Boş          => Dizin(Boş)   // boş kümenin tek altkümesi var: kendisi
  durum baş :: gerisi =>
    dez başsızlar = altKümeler(gerisi)          // başı ALMAYANLAR
    başsızlar ++ başsızlar.işle(ak => baş :: ak)  // ve başı ALANLAR
}

satıryaz(toplam(Dizin(3, 1, 2)))       // 6
satıryaz(altKümeler(Dizin(3, 1, 2)).boyu)  // 8 = 2³
```

`altKümeler`'i kardeş kitapçıkla karşılaştırın: orada "al ya da alma" kararını bir gezi kalıbıyla, geri alma satırlarıyla yazmıştık. Burada karar iki yarımın birleşimi olarak *tanımlanıyor*: başı almayan altkümeler, bir de başı eklenmiş hâlleri. Geri alacak bir şey yok, çünkü kimse bir şeyi değiştirmedi. İşlevsel üslubun özyinelemeyle arası işte bu yüzden çok iyi.

IV.5

Unutkan işlevi hızlandırmak

Özyinelemenin tuzağına gelme sırası bizde. Fibonaççi'yi tanımıyla yazalım:

tanımı yazdık, oldu

```
tanım fibYavaş(n: Sayı): Uzun =
  eğer (n < 2) n
  yoksa fibYavaş(n - 1) + fibYavaş(n - 2)
```

Doğru — ama `fibYavaş(45)` deyin ve çayınızı koyun. Sağdaki iki çağrı ikiye, dörde, sekize çatallanıyor ve aynı değerler milyonlarca kere yeniden hesaplanıyor. Çare, kardeş kitapçıktakiyle birebir aynı: **hesapladığını bir kenara yaz** (*memoizasyon*). Kenar defterimiz bir eşlem:

fib.kojo — BELLEKLİ SÜRÜM

```
dez bellek = Eşlem[Sayı, Uzun]()

tanım fibHızlı(n: Sayı): Uzun =
  eğer (n < 2) n
  yoksa bellek.alYoksa(n, {
    dez sonuç = fibHızlı(n - 1) + fibHızlı(n - 2)
    bellek.eşEkle(n -> sonuç)
    sonuç
  })

satıryaz(fibHızlı(45))    // 1134903170, göz açıp kapayana kadar
satıryaz(fibHızlı(90))    // 2880067194370816120 -- Uzun'a sığıyor, ucu ucuna
```

`alYoksa(n, …)` şunu diyor: *defterde n varsa onu ver; yoksa şu bloğu çalıştır.* Blok da hesaplayıp deftere yazıyor. `fibYavaş(45)` için çağrı sayısı iki milyarı geçer; `fibHızlı(45)` doksan küsur çağrıyla biter. Aradaki fark, Bölüm III'teki tabloların diliyle: milyonlarca kat — ve tek kuruşluk fikir: *aynı işi iki kere yapma.*

Bu fikrin bir adı ve koca bir bölümü var: dinamik programlama. Bölüm VI'da oraya döneceğiz.

TAKIM İŞİ

Biriniz `fibYavaş(40)`'ı başlatsın; süre işlerken öteki `fibHızlı`'yı yazsın. Hangisi önce bitirir? Bu yarış “algoritma seçimi” dersinin tamamıdır ve unutulmaz.

ALIŞTIRMALAR

1. **Eğrelti otu.** Ağaç işlevine üçüncü bir dal ekleyin ve açılarla oynayın. Sonra dalların boyuna göre kalem rengini yeşilden kahverengiye değiştirin. En güzel ağacı kim çizecek?

2. **Kare tanesi.** Koch adımındaki "çatı"yı üçgen yerine kare yapın: `sol(90); ileri; sağ(90); ileri; sağ(90); ileri; sol(90)` gibi. Ortaya çıkan şekle bakın; derinlik 3'te kaç çizgi çizildi?

3. **Sierpinski üçgeni.** Bir üçgen çizin; köşelerine üç yarı boyutlu üçgen çizin; onların da köşelerine... Taban ve adımı kendiniz kurun. İPUCU `zıpla` ve `konumVeYönüBelleğeYaz` / `konumVeYönüGeriYükle` işinizi kolaylaştırır.

4. **Permütasyonlar.** `altKümeler`'in kardeşi `permütasyonlar(d: Dizin[Sayı])`'ı yazın: bütün sıralanışlar. İPUCU Başı sırayla her öge yapıp gerisinin permütasyonlarının önüne ekleyin. `Dizin(1,2,3)` için 6 tane çıkmalı — `belirt` ile sınayın.

5. **Kümeyi ikiye bölün.** CSES Apple Division: n elmayı iki gruba ayırın, ağırlık farkı en az olsun. `altKümeler` tam bu iş için. `n ≤ 20` sınırına dikkat: 2^{20} bir milyon — bilgisayarınıza göre çerez. DEVAMI Aynı soruyu kardeş kitapçığın IV. bölümündeki gibi C++ ile de çözüp iki kodu yan yana koyun.

BÖLÜM V

Çizgeler ve Gezintiler

Şehirler ve yollar, bilgisayarlar ve kablolar, insanlar ve arkadaşlıklar — hepsi aynı matematiksel nesne: **çizge**. Kardeş kitapçık bu konuyu beş satırlık bir gezi kalıbıyla işler; biz aynı kalıbı bir de işlevsel kılıkta göreceğiz: değiştirmeden, geri almadan, kanıtı kendiliğinden gelen bir gezi.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

V.1

Çizge nedir, nasıl saklanır?

Bir **çizge** (*graph*) iki şeyden ibaret: **düğümler** (*vertex*) ve onları birleştiren **bağlar** (*edge*). Kuramın doğum günü bile belli: 1736'da Euler, Königsberg'in yedi köprüsünün hepsinden tam bir kere geçen yürüyüş olmadığını kanıtlarken köprü boylarının, ada büyüklüklerinin hiç önemli olmadığını, önemli olanın yalnızca *neyin neye bağlı olduğu* olduğunu gösterdi.

"Neyin neye bağlı olduğu" sözünü Bölüm II'de bir kapla söylemiştik: eşlem. Her düğümü komşularının kümesine eşleyelim; Koco'nun `Eşlem`'i tam bu iş için. (Dikkat: `Eşlem` *değişebilir* bir eşlemdir, `eşEkle` onu yerinde değiştirir. Bölüm II'nin değişmezlik ilkesine uyan kardeşi `Eşlek`; ona `+` ile eklenir ve yenisi döner. Çizgeyi bir kere kurup bir daha değiştirmeyeceğiz, `Eşlem` yeter.)

çizge.kojo — KOMŞULUK KÜMESİ

```
tür Düğüm = Harf
tür Çizge = Eşlem[Düğüm, Küme[Düğüm]]   // düğümden komşularına

dez ç: Çizge = Eşlem(
  'A' -> Küme('B', 'C'),
  'B' -> Küme('D'),
  'C' -> Küme('D', 'E'),
  'D' -> Küme('F'),
  'E' -> Küme('F'),
  'F' -> Küme('G'),
  'G' -> Küme[Düğüm]()          // çıkışı olmayan düğüm
)
```

`tür` satırları yeni bir şey *yapmıyor*, sadece ad takıyor — ama fark büyük: `Eşlem[Harf, Küme[Harf]]` yerine `Çizge` okuyunca kod problemin dilinde konuşuyor. Kardeş kitapçıktaki `using Cizge = map<Nokta, set<Nokta>>` satırının buradaki ikizi bu. Kâğıda çizin: A'dan iki yol çatallanıp D ile E'de buluşuyor, F'te birleşip G'de bitiyor.

> **PÜF NOKTASI**
>
> Çizgenin kendisi de bir işlev gibi kullanılır: `ç('A')` size A'nın komşularını verir. Küme de öyleydi: `komşular('D')` "D komşu mu?" demek. Eşlem, küme, işlev — işlevsel dünyada hepsi aynı kapıya çıkar: *girdi ver, çıktı al*.

V.2

Derinlemesine gezi, iki kılıkta

Önce yeni bir aletle tanışın. `indirge` ögeleri birleştirirken başlangıç değeri alamıyordu; **soldanKatla** (*fold left*) alır: bir başlangıç değeri, bir de "eldeki sonuçla sıradaki ögeyi birleştir" işlevi verirsiniz:

soldanKatla

```
Dizin(3, 1, 4).soldanKatla(0)((toplam, öge) => toplam + öge)   // 8
Dizin(3, 1, 4).soldanKatla("")((yazı, öge) => yazı + öge)     // "314"
```

Şimdi gezi. Amaç: bir düğümden başlayıp ulaşılabilen *bütün* düğümleri bulmak. İşlevsel kılıkta durum — gezilenler kümesi — değiştirilmez; her çağrı **yeni bir küme verir**:

gez.kojo — İŞLEVSEL DERİNLEMESİNE GEZİ

```
tanım gez(ç: Çizge, buradan: Düğüm, gezilen: Küme[Düğüm]): Küme[Düğüm] =
  eğer (gezilen(buradan)) gezilen            // TABAN: zaten uğramışız
  yoksa ç(buradan).soldanKatla(gezilen + buradan)(
    (g, komşu) => gez(ç, komşu, g)          // her komşudan derine in
  )

satıryaz(gez(ç, 'A', Küme[Düğüm]()))
// KıymaKüme(E, F, A, G, B, C, D) -- yedi düğümün yedisi de bulundu;
// sıralanış her seferinde değişebilir, çünkü kıyma (hash) kümesi sıra tutmaz
```

Dört satır. Okunuşu: *buraya uğramışsak elimizdeki küme neyse odur; uğramamışsak kendimizi ekleyip her komşunun getirdiklerini üst üste katlarız.* Geri alma satırı yok, işaretleme yok, silme yok — çünkü hiçbir şey değişmiyor. Bu kodun "bir düğümü iki kere sayar mı?" sorusuna yanıtı, kümenin tanımından çıkıyor: küme, tekrar tutmaz. **Kanıtın yarısını veri yapısı yapıyor.**

Aynı gezi, yığınla — ve kuyruk numarası

Şimdi kardeş kitapçığın en sevdiğim numarası, burada da aynen çalışıyor. Geziyi özyineleme yerine bir **yığınla** yazalım:

gez2.kojo — YIĞIN MI, KUYRUK MU?

```
tanım gez2(ç: Çizge, ilk: Düğüm) = {
  dez bekleyen = Yığın[Düğüm]()       // yığın: son giren ilk çıkar
  den gezilen = Küme[Düğüm]()
  bekleyen.koy(ilk)
  yineleDoğruysa(bekleyen.tane > 0) {
    dez şimdiki = bekleyen.al()   // "bu" diyemezdik: anahtar sözcük (this)
    eğer (!gezilen(şimdiki)) {
      gezilen = gezilen + şimdiki
      yaz(s"$şimdiki ")
      ç(şimdiki).herbiriİçin(komşu => bekleyen.koy(komşu))
    }
  }
}
```

Şimdi `Yığın` yerine `Kuyruk` koyun (`koy` yerine `ekle`, `al` yerine `baştanAl`). Tek kelimelik bu değişiklik geziyi **derinlemesineden** (*DFS*) **enlemesineye** (*BFS*) çevirir: yığın hep en son bulduğu yola dalar, kuyruk ise düğümleri başlangıca *uzaklık sırasıyla* işler. Aynı iskelet, bambaşka iki algoritma. İkisini de çalıştırdım; aynı çizge, iki farklı sıra:

çıktı

```
yığınla  : A C E F G D B   // C'ye dalıp ta G'ye kadar gitti
kuyrukla : A B C D E F G   // önce bütün komşular: uzaklık sırası
```

GERÇEK BİR HATA

`if (!gezilen(bu))` denetimini silerseniz program çoğu çizgede yine çalışır — ama iki yoldan ulaşılan düğümler iki kere işlenir, çevrimli bir çizgede ise hiç durmaz. Kardeş sınıfta bu hatayı C++ yazarken yaptık ve ancak çizgeye yedinci düğümü ekleyince fark ettik. Çizge kodunda ilk denetleyeceğiniz soru: *"buraya daha önce geldim mi?"* soruluyor mu?

V.3

Katman katman: en az adım

Enlemesine gezinin asıl armağanı şudur: başlangıçtan her düğüme **en az kaç adımda** gidileceğini söyler. İşlevsel kılıkta bunu katmanlarla anlatmak çok doğal: 0. katman başlangıç, 1. katman onun komşuları, 2. katman onların henüz görülmemiş komşuları...

katmanlar.kojo — İŞLEVSEL ENLEMESİNE GEZİ

```
tanım katmanlar(ç: Çizge, sınır: Küme[Düğüm],
                gezilen: Küme[Düğüm]): Dizin[Küme[Düğüm]] =
  eğer (sınır.boşMu) Boş                        // TABAN: yeni düğüm kalmadı
  yoksa {
    dez sonraki = sınır.düzİşle(d => ç(d))      // sınırın bütün komşuları...
                       .ele(k => !gezilen(k))   // ...görülmemiş olanları
    sınır :: katmanlar(ç, sonraki, gezilen ++ sonraki)
  }

dez kat = katmanlar(ç, Küme('A'), Küme('A'))
kat.ikileSırayla.herbiriİçin { durum (katman, uzaklık) =>
  satıryaz(s"$uzaklık adım: $katman")
}
```

çıktı

```
0 adım: Küme(A)
1 adım: Küme(B, C)
2 adım: Küme(D, E)
3 adım: Küme(F)
4 adım: Küme(G)
```

`düzİşle` (*flat map*) burada ilk kez iş başında: her düğümü komşu *kümesine* çevirip hepsini tek kümede düzlüyor. Ve neden "en az adım" sorusunun yanıtı bu? Çünkü bir düğüm hangi katmanda *ilk kez* görünüyorsa, oraya daha kısa yol olamaz — olsaydı önceki bir katmanda görünürdü. Kardeş kitapçık bu kanıtı kuyruklu kodun üstüne kurar; burada kanıt, kodun *şekline* gömülü.

Kaç parça?

Çizge kopuksa tek gezi her yere ulaşamaz. Bütün düğümleri sırayla deneyip gezilmemiş her birinden yeni bir gezi başlatırsak, kaç gezi başlattıysak çizge o kadar **bağlı parçadır**:

bağlı parçalar

```
dez düğümler = ç.dizine.işle { durum (d, _) => d }.sıralı

dez (parçaSayısı, _) =
  düğümler.soldanKatla((0, Küme[Düğüm]())) {
    durum ((sayı, gezilen), d) =>
      eğer (gezilen(d)) (sayı, gezilen)          // eski bir parçanın içinde
      yoksa (sayı + 1, gez(ç, d, gezilen))       // yeni parça keşfedildi
  }

satıryaz(s"$parçaSayısı parça")     // 1 parça: A'dan her yere ulaşılıyor
```

Yine `soldanKatla`, bu sefer taşıdığı şey bir demet: (şimdiye kadarki parça sayısı, şimdiye kadar gezilenler). Döngü + iki değişkenle yazacağınız her şeyi, başlangıç değeri + birleştirme işleviyle de yazabilirsiniz — bu, işlevsel programlamanın büyük değiş tokuşudur.

NEDEN SIRALADIK?

`düğümler`'in sonundaki `sıralı`'yı silin ve birkaç kere çalıştırın: bazen 1, bazen 2 parça çıkar! Çünkü eşlemin veriliş sırası her seferinde değişebilir ve bizim çizge **yönlü**: A'dan G'ye yol var ama G'den A'ya yok. Geziye A'dan başlarsanız tek parça görürsünüz; G'den başlarsanız G tek başına bir "parça" olur. Bunu ilk çalıştırmada değil, ancak çıktıyı sınayınca fark ettik — bağlı parçalar aslında *yönsüz* çizgelerin kavramı. Yönlü çizgede ne anlama geldiğini düşünmek güzel bir tartışma sorusu.

V.4

Yollar eşit değilse

Bütün bağların eşit sayıldığı dünyada "en kısa yol" demek "en az adım" demekti ve kuyruk yetiyordu. Bağların ağırlığı olunca — kilometre, dakika, ücret — kuyruğun yerini **öncelik sırası** alır ve algoritmanın adı **Dijkstra** olur. Koco'da `ÖncelikSırası` hazır; iskelet de gördüğünüz gezilerle aynı: *bekleyenlerden birini al, işle, komşularını beklet.* Değişen tek şey, "hangisini önce alacağız?"

Bu kitapçıkta Dijkstra'yı yazmıyorum; bilerek. Tam kodu, kanıtıyla ve tuzaklarıyla birlikte kardeş kitapçığın V. bölümünde bulacaksınız. Buradaki alıştırmanız onu Koco'ya çevirmek — ve çeviri yaparken iki dilde de aynı kalan çekirdeği görmek. O çekirdek, algoritmanın kendisidir; gerisi kılık.

TAKIM İŞİ

Çizgeleri konuşmanın en iyi yeri tahta değil, tuval. Kaplumbağayla çizge çizen bir `düğümÇiz(ad: Düğüm, x: Kesir, y: Kesir)` işlevi yazın: `atla(x, y)`, küçük bir `daire`, yanına `yazı(ad)`. Biriniz düğümleri, öteki bağları çizsin; yukarıdaki yedi düğümlü çizgeyi ekrana alın. Gezilerinizi bu resmin üstünde parmakla izleyin — kod başka, çizge başka şeymiş gibi durmasın.

ALIŞTIRMALAR

1. **Kuyruk sürümü.** `gez2`'yi kuyrukla yazın ve iki çıktıyı yan yana koyun. Hangi sıra hangi geziye ait, bakar bakmaz söyleyebiliyor musunuz?

2. **Yol var mı?** `ulaşırMı(ç, a, b): İkil` işlevini yazın. İpucu: `gez` zaten her şeyi biliyor; tek satır yeter.

3. **İki takım.** Sınıfı iki takıma ayırın; arkadaş olan ikili aynı takıma düşmesin. Gezi kalıbına "takım" bilgisi ekleyin: komşuya hep karşı takımı verin, çelişki çıkarsa `yanlış`. Hangi çizgelerde imkânsız? İPUCU Tek sayıda bağdan oluşan çevrim.

4. **Düğüm çözme oyunu.** Ders notlarımızdan iki bulmaca: Kocoyla düğüm çözme ve çözülemeyen bir düğüm. İkincisinin neden düzlemsel olamayacağını tartışın. (İkisinin de kodu ders deposunda, `kitapcik-koco/kod/` dizininde.)

5. **Dijkstra'yı Koco'ya çevirin.** Kardeş kitapçıktaki Dijkstra'yı `ÖncelikSırası` ile buraya taşıyın. PÜF Önceliği `(süre, şehir)` demetiyle verin; demetler soldan karşılaştırılır — C++'taki `pair` gibi.

BÖLÜM VI

Dinamik Programlama

Korkutucu adlı, bir cümlelik fikir: **büyük problemi kendinden küçük parçalarına böl, her parçayı bir kere çöz, bir daha çözme.** İşlevsel üslupta bu fikir en doğal hâline kavuşur, çünkü tümevarım formülü zaten bir işlev tanımıdır. Bölümün sonunda kitapçığı, karmaşık sayıların içinden geçip yola devam ederek kapatacağız.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

VI.1

Aslında bunu zaten yaptınız

Bölüm IV'te Fibonaççi'ye bir kenar defteri ekleyip milyonlarca kat hızlandırmıştık. O, dinamik programlamaydı. Adı büyük, yaptığı iş bu kadar. İki yazılış biçimi var, ikisi de gerekli:

Yukarıdan aşağı (YA)
Özyinelemeli tanım + kenar defteri. Tümevarım formülünü doğrudan koda çevirirsiniz; **düşünmesi ve kanıtlaması kolay**.

Aşağıdan yukarı (AY)
En küçük durumdan başlayıp bir çizelgeyi doldurursunuz. Özyineleme yok; **genelde daha hızlı**, belleği kısmak mümkün.

Kardeş kitapçığın öğüdü burada da geçerli: **YA ile düşün, AY ile yaz.** Biz ikisini de aynı problemin üstünde deneyeceğiz.

VI.2

Izgarada kaç yol var?

Project Euler 15: 20×20'lik ızgaranın sol üst köşesinden sağ alt köşesine, yalnızca sağa ve aşağıya giderek kaç yol var? Tümevarım tek cümle: bir kareye ya üstten ya soldan gelinir, öyleyse `yol(r, c) = yol(r−1, c) + yol(r, c−1)`. Bu cümle, Fibonaççi'deki kalıbın birebir aynısıyla koda dönüşür:

yollar.kojo — YUKARIDAN AŞAĞI

```
dez bellek = Eşlem[(Sayı, Sayı), Uzun]()

tanım yol(r: Sayı, c: Sayı): Uzun =
  eğer (r == 0 || c == 0) 1          // TABAN: kenar boyunca tek yol var
  yoksa bellek.alYoksa((r, c), {
    dez s = yol(r - 1, c) + yol(r, c - 1)
    bellek.eşEkle((r, c) -> s)
    s
  })

satıryaz(yol(20, 20))     // 137846528820
```

Defterin anahtarı bu sefer bir demet: `(r, c)`. Kenar defteri olmadan bu hesap 20×20'de asla bitmezdi; defterle, her kare bir kere hesaplanır: 400 kayıt, o kadar. (21×21 = 441 kare var; kenardaki kareler taban durumu, deftere hiç yazılmıyor.)

Aşağıdan yukarı, işlevsel çizelge

Şimdi aynı işi çizelgeyle yapalım. Kardeş kitapçıkta buranın yıldızı şu tek satırdı: `satir[c] += satir[c-1]` — çizelgenin tamamı yerine tek bir satırı tutup üstüne yaza yaza ilerlemek. O satırın yaptığı işin bir adı var: **birikimli toplam**. İşlevsel ikizi de şöyle:

yollar2.kojo — AŞAĞIDAN YUKARI

```
// (1, 1, 1) -> (1, 2, 3) -> (1, 3, 6): her satır bir öncekinin birikimlisi
tanım birikimli(satır: Dizin[Uzun]): Dizin[Uzun] =
  satır.kuyruğu.soldanKatla(Dizin(satır.başı)) {
    (bina, öge) => (bina.başı + öge) :: bina
  }.tersi

den satır = (0 |-| 20).dizine.işle(k => 1L)   // ilk satır: hep 1  (L: Uzun sabiti)
yinele(20) { satır = birikimli(satır) }
satıryaz(satır.sonu)     // 137846528820 -- aynı yanıt
```

Neden çalışıyor? Izgara çizelgesinde her satır, bir üst satırın birikimli toplamıdır — kâğıda 3×3'lük bir çizelge çizip yukarıdaki yorum satırındaki diziyi elle üretin, kendi gözünüzle görün. İki sürüm de aynı yanıtı veriyor; biri tanımı, öteki çizelgeyi anlatıyor. **Bir problemi iki biçimde yazabilmek, onu gerçekten anlamış olmanın en iyi göstergesidir.**

VI.3

En az kaç bozuk para?

Paralarımız 1, 5 ve 7 kuruş; 11 kuruşu en az kaç parayla öderiz? Sezgi "en büyükten başla" der: 7+1+1+1+1, **beş para**. Oysa 5+5+1 **üç para** eder — aceleci (*greedy*; birebir çevirisiyle "açgözlü") yöntem burada yanılır. Acele işe şeytan karışır. Doğrusu, son parayı sormaktır: *son koyduğum para hangisiydi?* Hangisiyse geriye daha küçük ama aynı türden bir problem kalır:

tümevarım

```
f(0) = 0
f(x) = 1 + enUfağı( f(x - 1), f(x - 5), f(x - 7) )
f(x) = ∞    (x < 0 için: "böyle ödeme olmaz")
```

bozukpara.kojo — FORMÜLÜN BİREBİR ÇEVİRİSİ

```
dez paralar = Dizin(1, 5, 7)
dez defter = Eşlem[Sayı, Sayı]()

tanım enAzPara(tutar: Sayı): Sayı =
  eğer (tutar == 0) 0
  yoksa eğer (tutar < 0) 1000000      // "sonsuz": enUfağı bunu asla seçmez
  yoksa defter.alYoksa(tutar, {
    dez s = 1 + paralar.işle(p => enAzPara(tutar - p)).enUfağı
    defter.eşEkle(tutar -> s)
    s
  })

satıryaz(enAzPara(11))     // 3
```

Ortadaki satırı formülle yan yana koyun: `1 + paralar.işle(p => enAzPara(tutar - p)).enUfağı` tam olarak “1 + min f(x−p)” diye okunuyor. İşlevsel üslubun dinamik programlamayla arası bu yüzden çok iyi: **tümevarım formülü zaten bir işlev tanımı**; çeviri kaybı sıfır. “Sonsuz” için kocaman bir sayı kullanmak da kardeş kitapçıktan tanıdığınız numara: `enUfağı` onu kendiliğinden eler, biz de eksi tutarlar için ayrı ayrı koşul yazmaktan kurtuluruz.

SIK YANILGI

Defteri unutmak. `defter.eşEkle` satırını silerseniz program yine doğru yanıt verir; ama 11 yerine 40 istediğinizde bir daha dönmez. Bellekle hızlandırma gitti, üstel özyineleme geri geldi. “Sonsuz” için de kocaman ama *taşmayan* bir sayı seçin: `1 + Int.MaxValue` eksiye döner ve `enUfağı` onu sevinçle seçer.

TAKIM İŞİ

Beş adımlık DP reçetesi — fonksiyonu yaz, tabanları belirle, formülü kur, kanıtla, kodla — kardeş kitapçığın VI. bölümünde ayrıntısıyla duruyor. Takımca şunu yapın: reçeteyi oradan okuyun ve bu bölümdeki iki probleme adım adım uygulayarak kâğıda dökün. Kanıt adımında sorulacak iki soru hep aynı: formül bütün durumları kapsıyor mu? Hiçbirini iki kere saymıyor mu?

Kendi türünüz: karmaşık sayılar

Kitapçığı kapatmadan bir borcumuz var: kapaktaki Mandelbrot kümesi. Onun için karmaşık sayılar gerekiyor — ve bu, kendi türünüzü yazmayı öğrenmenin tam sırası. Koco'da bunun aleti `durum sınıf` (*case class*):

karmaşık.kojo — KENDİ TÜRÜNÜZ

```
durum sınıf Karmaşık(g: Kesir, s: Kesir) {     // gerçek ve sanal kısım
  tanım +(öbürü: Karmaşık) = Karmaşık(g + öbürü.g, s + öbürü.s)
  tanım *(öbürü: Karmaşık) =
    Karmaşık(g * öbürü.g - s * öbürü.s, g * öbürü.s + s * öbürü.g)
  tanım büyüklüğü = karekökü(g * g + s * s)
}

dez a = Karmaşık(1, 2)
satıryaz(a + a)       // Karmaşık(2.0,4.0)
satıryaz(a * a)       // Karmaşık(-3.0,4.0)
```

`+` ve `*` sıradan yöntem adları — Scala'da işleçler de birer yöntemdir, C++'taki `operator+` gibi ama daha az törenle. İki satır tanımla, kendi türünüz sayılar kadar rahat toplanıp çarpılıyor.

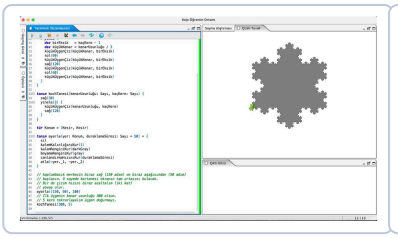
Mandelbrot kuralı tek satır: `z ← z² + c`. Bir `c` noktası için sıfırdan başlayıp bu kuralı döndürün; `z` büyüyüp kaçarsa `c` kümenin dışında, yüz adımda kaçmazsa (bizim kararımızla) içinde:

kaçış adımı

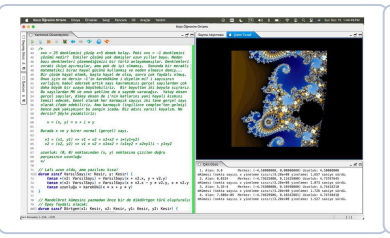
```
tanım kaçışAdımı(c: Karmaşık): Sayı = {
  tanım dön(z: Karmaşık, adım: Sayı): Sayı =
    eğer (adım == 100 || z.büyüklüğü > 2) adım
    yoksa dön(z * z + c, adım + 1)
  dön(Karmaşık(0, 0), 0)
}

satıryaz(kaçışAdımı(Karmaşık(0, 0)))       // 100: hiç kaçmadı, küme içinde
satıryaz(kaçışAdımı(Karmaşık(1, 1)))       // 2: iki adımda kaçtı
```

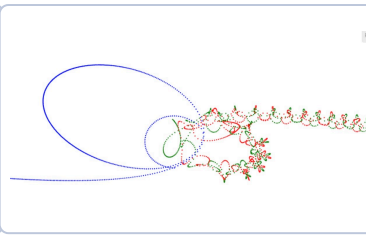
Resmi çizmek için tuvalin her noktasını bir karmaşık sayıya çevirip `kaçışAdımı`'na göre renklendirmek kalıyor: erken kaçan noktaya açık, geç kaçana koyu bir renk. Bu son adımı **bilerek yazmıyorum** — iskeleti kurdunuz, boyası sizin. Kaplumbağayla `atla` + `nokta` ikilisi iş görür; sabırsızlananlar için tam sürüm Koco'nun örnekleri arasında da var.



Koch tanesi. Bölüm IV'te yazdınız. Buradaki sürüm renkli ve derinliği klavyeden ayarlanıyor — sizinkine eklemesi kolay.



Mandelbrot kümesi. Yukarıdaki `kaçışAdımı`'nın boyanmış hâli. Kenarına yaklaştıkça yeni desenler açılır; sınırı hem var hem yok.



Üç cisim. Birbirini çeken üç gök cismi. Kalem kâğıtla kapalı çözümü *yok*; tek yol adım adım benzetim — yani `dön(z, adım + 1)` kalıbının fizikteki kardeşi.

SON SÖZ

Yola devam

Altı bölüm önce `ileri(100)` yazmıştınız; bugün değişmez kümelerle çizge geziyor, tümevarım formüllerini koda çeviriyorsunuz. Yol buradan üç kola ayrılıyor, üçü de açık:

Koco'da derinleşin

iKojo'nun Türkçe kılavuzunu izleyin; oyunlar, canlandırmalar, müzik — hepsi bu kitapçıktaki araçlarla yazılıyor.

Kardeş kitapçığa geçin

Aynı kavramların C++ hâli, yarışma dünyasının kapısı: Keyifli Bir Başlangıç. Buradan sonra orası çok daha kolay gelecek.

Soru çözün

Project Euler işlevsel üsluba bayılır; çoğu soru bir `işle` / `ele` / `indirge` zinciridir. CSES ile de algoritma kasları çalışır.

Bir şey inşa edin

Bir oyun, bir benzetim, bir fraktal galerisi. Yarışma kodu bir günde ölür; proje kodu sizinle kalır — ve Koco projeleri gösterişlidir, arkadaşlarınıza izletin.

Son üç cümle

Bu kitapçık boyunca tuzakları saklamadım: taşan sayılar, durmayan özyinelemeler, iki kere sayılan düğümler, aceleciliğin yanıldığı paralar. Hepsiyle kardeş sınıfta gerçekten karşılaştık ve hepsini birlikte bulduk. **Hata yapmak öğrenmenin yan etkisi değil, yöntemidir.**

Takıldığınızda yalnız kalmayın: bir arkadaş, bir kâğıt, küçük bir örnek. Bu iki kitapçığın en çok tekrar ettiği tavsiye buydu; tesadüf değil.

Ve acele etmeyin. Kaplumbağayla başladık; kaplumbağa gibi bitirelim: yavaş, kararlı ve iz bırakarak. Kolay gelsin, keyifli ve öğretici olsun.

SON ALIŞTIRMALAR

1. **Zar dizileri.** Bir zarı defalarca atıyoruz; toplamı n olan kaç farklı zar dizisi var? (CSES DP 1.) Formül: `f(n) = f(n−1) + … + f(n−6)`, `f(0) = 1`. Bu bölümdeki kalıpla, `işle` ve `topla` kullanarak yazın.

2. **Hangi paralar?** `enAzPara` sayıyı veriyor ama paraların dökümünü vermiyor. `Dizin[Sayı]` döndüren sürümünü yazın: `enAzPara(11)` yerine `Dizin(5, 5, 1)` gibi. İPUCU `enUfağı(d => d.boyu)`.

3. **Izgarada engel.** Yol sayma problemine yasak kareler ekleyin: yasak karede `yol(r, c) = 0`. Tümevarımın tek satırı değişiyor — hangisi?

4. **Mandelbrot'u boyayın.** VI.4'teki iskeleti bitirip resmi çizin. Sonra `yaklaş` komutuyla kenarına dalın. En güzel köşenin ekran görüntüsünü sınıfla paylaşın.

5. **Kendi kitapçığınızı yazın.** Bu kitapçıktan öğrendiğiniz bir konuyu, bir arkadaşınıza anlatacak biçimde iki sayfa yazın: çalışan kod, gerçek çıktı, bir de alıştırma. Bir şeyi anlatabildiğiniz gün onu gerçekten öğrenmiş olursunuz. EN ÖNEMLİSİ

Bu kitapçık, 2024–26 dönemlerindeki derslerimizden, Koco'nun Türkçe sürümü üzerindeki gönüllü çalışmalardan ve kardeş kitapçık *Programlamaya ve Algoritmalara Keyifli Bir Başlangıç*'tan damıtıldı. Ders notlarının tamamı GitHub deposunda.

SÖZLÜK

Küçük Sözlük

Kitapçıkta geçen terimlerin Türkçesi, Scala'daki ya da İngilizcedeki karşılığı ve bir cümlelik anlamı; yanında ilk geçtiği bölüm. Koco'nun Türkçe adlarıyla Scala'nın özgün adlarını yan yana görmek için de bakın. Daha geniş ve aranabilir sürümü, her kavramın alıştırmasıyla birlikte, Keyifli Kavram Dizini'nde.

Kılavuz | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama | Sözlük

A – D

A'dan D'ye

Aceleci algoritma *greedy algorithm* — VI

Her adımda o an en iyi görüneni seçmek. 11 kuruşu 7+1+1+1+1 diye ödeyen yöntem; doğrusu 5+5+1.

Alt problem *subproblem* — IV, VI

Büyük problemin aynı biçimdeki küçük hâli. Küçük ağaç büyük ağacın dalı; kısa sarmal uzun sarmalın içi.

Aşağıdan yukarı *bottom-up* — VI

DP çizelgesini küçük durumlardan büyüğe doldurmak; işlevsel üslupta katlamayla (`indirge`) yazılır.

Bağ *edge* — V

Çizgede iki düğümü birleştiren çizgi. Komşuluk eşleminde her düğümün karşısındaki küme, bağlarının öteki uçları.

Bağlı parça *connected component* — V

Birbirine bir yolla ulaşılabilen düğümlerin kümesi. Gezi işlevini gezilmemiş her düğümden bir kere başlatınca sayılır.

Baş ve kuyruk *head and tail* II

Dizinin ilk ögesi ve gerisi: `başı`, `kuyruğu`. Her dizin ya `Boş` tur ya da bir baş ve gerisi.

Belki *Option* II

“Var mı, yok mu?” sorusunun değer hâli: ya `Biri(6)` ya `Hiçbiri`. Eşlemde `al` bunu verir.

Bellekle hızlandırma *memoization* IV, VI

Hesaplanan sonucu bir eşleme (*deftere*) yazıp bir daha hesaplamamak. `alYoksa` ile tek deyişte.

Birim *Unit* III

“Değer vermiyorum” türü. Çizen işlevlerin çıktı türü; C++'taki `void`'in akrabası.

Çizge *graph* V

Düğümler ve aralarındaki bağlar. Koco'da `Eşlem[Düğüm, Küme[Düğüm]]` olarak saklanır ve tuvale çizilebilir.

Değişmezlik *immutability* II

Bir kere kurulan değerin bir daha değişmemesi. `sıralı` yeni dizin verir, eskisini bozmaz; kimse veriyi arkanızdan değiştiremez.

Derinlemesine gezi *depth-first search, DFS* V

Bir komşuya dalıp gidebildiğin kadar gitmek. İşlevsel kılığı gezilen kümeyi girdi alıp yenisini döndürür; yığınlı kılığı da var.

Deyiş *expression* I

Değer üreten kod parçası. Scala'da `eğer` bile bir deyiş: `eğer (a > b) a yoksa b`.

dez, den *val, var* I

Değişmez değer ve değişken. Önce `dez` deneyin; `den`'e ancak mecbur kalınca dönün.

Dijkstra V

Ağırlıklı çizgede en kısa yol: enlemesine gezinin kuyruğu öncelik sırasıyla değişmiş hâli.

Dinamik programlama *dynamic programming, DP* VI

Örtüşen alt problemleri bir kere çözüp saklamak. Tümevarım formülü zaten bir işlev tanımıdır; çeviri kaybı sıfır.

Dizin *List* II

İşlevselin atardamarı: değişmez bağlı liste. `Dizin(3, 1, 4)`; masaüstü Koco `List(3, 1, 4)` diye yazdırır.

durum *case* II

Örüntü eşlemede bir dal: `durum (harf, kaç) =>` demeti yakalayıp parçalarına ad verir.

Düğüm *vertex, node* V

Çizgenin noktası. Kodda `Düğüm` türü çoğu zaman bir `Yazı` ya da `Sayı`.

E'den K'ye

eğer, yoksa *if, else* I

Koşul; değer üretir. `eğer (a = b)` yazarsanız Koco çalıştırmadan itiraz eder: `==` gerekir.

ele *filter* I, II

Her ögeye bir soru sorar, yanıtı `doğru` olanları tutar. Ögeler değişmez, boy kısalır.

Enlemesine gezi *breadth-first search, BFS* V

Katman katman: önce bütün komşular, sonra komşuların komşuları. En az adımı verir.

Eşlem *Map* II

Anahtardan değere: `Eşlem("Ankara" -> 6)`. Koco'daki `Eşlem` değişebilir (`eşEkle`); değişmez kardeşi `Eşlek`.

Fraktal *fractal* IV

Parçası bütününe benzeyen şekil: ağaç, Koch tanesi, Sierpinski üçgeni. Gözle görülür özyineleme.

Gezildi işareti *visited marker* V

Aynı düğüme iki kere girmemek için tutulan küme: `eğer (!gezilen(bu))`. Çizge kodunda ilk denetlenecek satır.

indirge *reduce, fold* II

Ögeleri ikişer ikişer birleştirip tek değere iner: `indirge((a, b) => a + b)`. Kısaltmaları `topla`, `çarp`, `enİrisi`.

iKojo Kılavuz

Koco'nun tarayıcıda çalışan sürümü: ikojo.fly.dev. Kurulum yok; tuvali her çalıştırmada kendiliğinden temizler.

işle *map* II

Her ögeye aynı işlevi uygular, sonuçların dizinini verir. Boy değişmez, ögeler dönüşür.

İşlevsel programlama *functional programming* II, III

Değişmez veri ve değer gibi elden ele verilen işlevlerle programlama. Döngülerin yarısı `işle` / `ele` / `indirge` zincirine döner.

Kaplumbağa *turtle* I

Tuvalde baktığı yöne yürüyen, yürürken çizen küçük robot. `ileri`, `sağ`, `sol`, `zıpla`.

Kesir *Double* I

Virgüllü sayı türü. `7 / 2` üç eder; `7.0 / 2` üç buçuk.

Komşuluk eşlemi *adjacency map* V

Her düğümün karşısında komşularının kümesi: çizgeyi saklamanın Koco'daki yolu.

Kuyruk, Yığın *Queue, Stack* II

İlk giren ilk çıkar / son giren ilk çıkar. Aynı gezi kodunda birini ötekiyle değiştirmek algoritmayı değiştirir.

Küme *Set* II

Tekrarsız, sırasız. Aynı zamanda bir işlev: `k(4)` "4 kümede mi?" diye sorar.

L – Z

L'den Z'ye

Lambda *anonymous function* II

Adsız işlev: `s => s * s`. "`s`'yi al, karesini ver." `işle`'ye böyle verilir.

Öncelik sırası *priority queue* II, V

En öncelikli öğeyi önce veren kap: `ÖncelikSırası`. Dijkstra'nın kuyruğu.

Öne ekleme *cons* II

`1 :: Dizin(2, 3)`: dizinin önüne öge ekleyip yeni dizin almak; eski dizin değişmez.

Örüntü eşleme *pattern matching* — II

Bir değeri biçimine göre ayırıp parçalarına ad vermek; `durum` ile yazılır.

Özyineleme *recursion* — IV

İşlevin kendini çağırması. Kendini çağıran tanıma çıktı türünü siz yazarsınız: `: Birim`, `: Sayı`.

Sayı, Uzun, İriSayı *Int, Long, BigInt* — I

Tam sayı türleri: iki milyara kadar, dokuz kentilyona kadar, sınırsız. Taşma burada da var; `İriSayı` taşmaz.

Taban durumu *base case* — IV

Özyinelemenin kendini çağırmadan durduğu en küçük hâl: `eğer (boy > 2)`. İlk yazılacak satır.

tanım *def* — I

İşlev tanımı: `tanım kare(kenar: Sayı) = { ... }`. Gövde bir deyiştir, değeri neyse çıktı odur.

Tuval *canvas* — I

Kaplumbağanın çizdiği alan. `sil()` temizler; iKojo her çalıştırmada kendiliğinden temizler.

Tümevarım formülü *recurrence* — VI

Büyük durumu küçüklerle anlatan eşitlik: `f(x) = 1 + enUfağı(f(x−1), f(x−5), f(x−7))`. Koco'da birebir koda çevrilir.

Yazı *String* — I

Karakter dizisi. `s"$harf -> $kaç"` yazı kalıbı; `ele` ve `öbekle` yazıda da çalışır.

yinele, yineleDizinli *repeat, repeatFor* — I

Bir bloğu *n* kere çalıştırmak; sayaçlısı `yineleDizinli(10) { k => ... }`. III. bölümde kendi `yinele`'nizi yazarsınız.

Yukarıdan aşağı *top-down* — VI

Bellekli özyinelemeyle DP: büyük tutardan başla, gerektikçe küçükleri çöz, deftere yaz.

Yüksek mertebeli işlev *higher-order function* — III

İşlev alan ya da işlev veren işlev: `dönence(12, 30) { çokgen(4, 100) }`, `işle`, `süresi`.